

GIÁO TRÌNH PHÁT TRIỂN ỨNG DỤNG WEB NÂNG CAO

Phiên bản V4 · .NET 10 + Next.js App Router

TÀI LIỆU ĐẶC TẢ YÊU CẦU PHẦN MỀM

Software Requirements Specification (SRS)

Tiêu chuẩn IEEE 830 / ISO/IEC/IEEE 29148:2018

Dự án: Blog Ẩm thực và Nấu ăn *Culinary Blog*

Phiên bản tài liệu	1.0.0
Ngày phát hành	04/06/2026
Trạng thái	Đã duyệt (Approved)
Công nghệ Backend	.NET 10 Minimal APIs, C#
Công nghệ Frontend	Next.js App Router, TypeScript
Cơ sở dữ liệu	PostgreSQL 16
Object Storage	MinIO (S3-Compatible)
Cache	Redis 7

Tài liệu này được biên soạn theo tiêu chuẩn IEEE 830 / ISO/IEC/IEEE 29148:2018.

LỊCH SỬ THAY ĐỔI TÀI LIỆU

Phiên bản	Ngày	Tác giả / Vai trò	Nội dung thay đổi	Trạng thái
1.0.0	04/06/2026	Senior BA / Architect	Phát hành lần đầu – Bản hoàn chỉnh theo IEEE 830 / ISO 29148.	Approved
0.9.0	20/05/2026	Senior BA	Bổ sung Chương 7 (Data Model), Chương 8 (API Spec) và Phụ lục.	Under Review
0.8.0	05/05/2026	Senior BA	Hoàn thiện Chương 3 (FR), bổ sung FR-FILE, FR-JOB, FR-OBS.	Draft
0.5.0	15/04/2026	Senior BA	Phác thảo ban đầu: Chương 1–4 (skeleton).	Draft

Phê duyệt tài liệu: Tài liệu phiên bản 1.0.0 đã được xem xét và phê duyệt bởi Trưởng nhóm Kiến trúc Hệ thống (Lead Systems Architect). Mọi thay đổi từ phiên bản 1.0.0 trở đi đều phải thông qua quy trình Change Request (CR) và được cập nhật vào bảng này.

MỤC LỤC

LỊCH SỬ THAY ĐỔI TÀI LIỆU	2
MỤC LỤC.....	3
CHƯƠNG 1. GIỚI THIỆU	6
1.1. Mục đích Tài liệu	6
1.2. Phạm vi Sản phẩm.....	6
1.2.1. Tên và Định danh	6
1.2.2. Mô tả Sản phẩm	6
1.2.3. Những gì KHÔNG thuộc phạm vi.....	7
1.3. Định nghĩa, Từ viết tắt và Ký hiệu	7
1.4. Tài liệu Tham chiếu.....	8
1.5. Tổng quan Tài liệu	10
CHƯƠNG 2. MÔ TẢ TỔNG QUAN HỆ THỐNG	11
2.1. Bối cảnh Sản phẩm.....	11
2.1.1. Vị trí trong Hệ sinh thái	11
2.1.2. Quan hệ với Hệ thống Ngoài.....	11
2.2. Chức năng Sản phẩm Tổng quát	12
2.3. Các Lớp Người dùng và Đặc điểm.....	12
2.4. Môi trường Vận hành	13
2.4.1. Môi trường Server (Production)	13
2.4.2. Môi trường Phát triển (Development)	13
2.4.3. Yêu cầu Trình duyệt Client.....	14
2.5. Ràng buộc Thiết kế và Hiện thực	14
2.6. Giả định và Phụ thuộc.....	15
2.6.1. Giả định.....	15
2.6.2. Phụ thuộc Bên ngoài	15
CHƯƠNG 3. YÊU CẦU CHỨC NĂNG CHI TIẾT	17
3.1. Module Xác thực và Quản lý Người dùng (FR-AUTH)	17
FR-AUTH-001: Đăng ký Tài khoản (User Registration).....	17
FR-AUTH-002: Đăng nhập bằng Email/Mật khẩu (Local Login)	18
FR-AUTH-003: Đăng nhập bằng Google OAuth 2.0	19
FR-AUTH-004: Làm mới Access Token (Token Refresh).....	20
FR-AUTH-005: Đăng xuất (Logout / Token Revocation).....	21
FR-AUTH-006: Xem Hồ sơ Cá nhân (View Profile).....	22
FR-AUTH-007: Cập nhật Hồ sơ Cá nhân (Update Profile)	23
3.2. Module Quản lý Danh mục (FR-CAT)	23
FR-CAT-001: Xem Danh sách Danh mục.....	23
FR-CAT-002: Xem Chi tiết Danh mục và Công thức.....	24
FR-CAT-003: Tạo Danh mục Mới [Admin].....	25

FR-CAT-004: Cập nhật Danh mục [Admin]	26
FR-CAT-005: Xóa Danh mục [Admin].....	26
3.3. Module Quản lý Công thức Nấu ăn (FR-RCP).....	27
FR-RCP-001: Xem Danh sách Công thức (Paginated + Filtered + Sorted)	27
FR-RCP-002: Xem Chi tiết Công thức	28
FR-RCP-003: Tạo Công thức Nấu ăn Mới [Author/Admin]	29
FR-RCP-004: Cập nhật Công thức [Author-Owner/Admin].....	30
FR-RCP-005: Xuất bản / Hủy Xuất bản Công thức	31
FR-RCP-006: Lưu trữ Công thức (Archive).....	32
FR-RCP-007: Xóa Công thức [Author-Owner/Admin]	32
FR-RCP-008: Quản lý Ảnh Công thức (Upload / Set Primary / Delete).....	33
FR-RCP-009: Quản lý Nguyên liệu (CRUD RecipeIngredient)	34
FR-RCP-010: Quản lý Các bước Thực hiện (CRUD RecipeStep).....	35
3.4. Module Tìm kiếm và Phân trang (FR-SRCH).....	36
FR-SRCH-001: Tìm kiếm Toàn văn bản (Full-Text Search).....	36
FR-SRCH-002/003/004: Lọc, Sắp xếp và Phân trang (Tóm tắt)	37
3.5. Module Quản lý Tập tin (FR-FILE)	37
3.6. Module Background Jobs (FR-JOB).....	38
3.7. Module Quan sát Hệ thống (FR-OBS).....	39
4. Yêu cầu Phi Chức năng (NFR).....	40
4.1. Hiệu năng (NFR-PERF)	40
4.2. Bảo mật (NFR-SEC)	41
4.3. Khả năng Sử dụng (NFR-USE).....	42
4.4. Độ tin cậy (NFR-REL).....	42
4.5. Khả năng Bảo trì (NFR-MAINT).....	43
4.6. Khả năng Mở rộng (NFR-SCALE).....	44
4.7. Tối ưu SEO (NFR-SEO)	44
5. Yêu cầu Giao diện Ngoài	46
5.1. Giao diện Người dùng (UI)	46
5.2. Giao diện Phần mềm – REST API	47
5.3. Giao diện Dịch vụ Bên thứ ba.....	47
5.4. Giao diện Phần cứng	48
6. Kiến trúc Hệ thống	50
6.1. Tổng quan Kiến trúc	50
6.2. Kiến trúc Backend – Clean Architecture.....	50
6.3. CQRS + MediatR Pipeline	51
6.4. Mô hình Quan hệ Thực thể (ERD tóm tắt)	52
6.5. Triển khai – Docker Compose	52
7. Mô hình Dữ liệu	54

7.1. BaseEntity (Abstract)	54
7.2. Recipe	54
7.2.1. RecipeNutrition (Owned Entity — cột trong bảng Recipes)	56
7.3. RecipeStep	57
7.4. RecipeIngredient	57
7.5. RecipeImage	58
7.6. Category	58
7.7. ApplicationUser (extends IdentityUser)	58
7.8. RefreshToken	59
8. Đặc tả REST API	61
8.1. Authentication Module (/auth)	61
8.2. Categories Module (/categories)	62
8.3. Recipes Module (/recipes)	63
8.4. Recipe Images (/recipes/{id}/images)	64
8.5. Recipe Steps (/recipes/{id}/steps)	64
8.6. Recipe Ingredients (/recipes/{id}/ingredients)	65
8.7. Health Check Endpoints	65
Phụ lục A – HTTP Status Codes	67
Phụ lục B – Application Error Codes	67
Phụ lục C – Từ điển Thuật ngữ	69

CHƯƠNG 1. GIỚI THIỆU

1.1. Mục đích Tài liệu

Tài liệu Đặc tả Yêu cầu Phần mềm (Software Requirements Specification – SRS) này được biên soạn theo tiêu chuẩn IEEE 830-1998 và ISO/IEC/IEEE 29148:2018 nhằm mô tả đầy đủ, chính xác và nhất quán toàn bộ yêu cầu chức năng (Functional Requirements) và yêu cầu phi chức năng (Non-Functional Requirements) của dự án ứng dụng web Blog Ẩm thực và Nấu ăn (Culinary Blog).

Tài liệu này phục vụ các đối tượng sau:

- Nhóm phát triển Backend (.NET 10/C#):** Căn cứ thiết kế API, domain model, và business rules.
- Nhóm phát triển Frontend (Next.js/TypeScript):** Căn cứ thiết kế giao diện, luồng người dùng và tích hợp API.
- Kỹ sư Kiểm thử (QA/QC):** Cơ sở xây dựng test cases, kiểm thử chấp nhận (acceptance testing).
- Kiến trúc sư Hệ thống:** Tham chiếu khi đưa ra quyết định kiến trúc (architecture decisions).
- Giảng viên và Sinh viên:** Tài liệu học thuật mẫu cho dự án thực hành xuyên suốt giáo trình.
- Stakeholder / Product Owner:** Phê duyệt phạm vi và ưu tiên tính năng.

Phạm vi hiệu lực: Tài liệu này có hiệu lực từ phiên bản 1.0.0 và là tài liệu nền tảng (baseline) cho toàn bộ vòng đời phát triển dự án. Mọi thay đổi yêu cầu sau khi tài liệu được phê duyệt phải tuân theo quy trình quản lý thay đổi (Change Management Process).

1.2. Phạm vi Sản phẩm

1.2.1. Tên và Định danh

Thuộc tính	Giá trị
Tên sản phẩm	Culinary Blog – Blog Ẩm thực và Nấu ăn
Định danh dự án	CULINARY-BLOG-V1
Loại hệ thống	Ứng dụng Web Full-Stack (API-Driven Architecture)
Phiên bản sản phẩm	1.0.0
Môi trường đích	Cloud/On-premise (Docker Compose + Nginx)

1.2.2. Mô tả Sản phẩm

Culinary Blog là một nền tảng web cho phép người dùng chia sẻ, khám phá và lưu trữ các công thức nấu ăn từ nhiều nền ẩm thực khác nhau. Ứng dụng cung cấp hệ sinh thái hoàn chỉnh bao gồm:

- Nền tảng chia sẻ công thức:** Tác giả (Author) đăng tải công thức với hình ảnh, danh sách nguyên liệu chi tiết, hướng dẫn từng bước thực hiện và thông tin dinh dưỡng.

- **Tổ chức nội dung:** Phân loại công thức theo danh mục (Category), độ khó (Difficulty Level), thời gian chuẩn bị và nấu.
- **Tìm kiếm thông minh:** Full-Text Search tiếng Việt sử dụng PostgreSQL tsvector/tsquery với unaccent extension.
- **Bảo mật đa lớp:** Xác thực JWT stateless, phân quyền theo vai trò (RBAC) và theo tài nguyên (Resource-Based Authorization), đăng nhập Google OAuth 2.0.
- **Tối ưu hiệu năng và SEO:** Redis distributed cache, Next.js ISR, Open Graph Protocol, JSON-LD Schema.org Recipe markup.
- **Quan sát hệ thống:** Structured logging (Serilog), distributed tracing (OpenTelemetry), health check endpoints.

1.2.3. Những gì KHÔNG thuộc phạm vi

Các tính năng sau đây nằm ngoài phạm vi phiên bản 1.0.0:

- Hệ thống bình luận (Comment System) và đánh giá sao (Rating System).
- Tính năng lưu/đánh dấu công thức yêu thích (Bookmark/Favorite).
- Thông báo real-time (SignalR/WebSocket).
- Ứng dụng di động native (iOS/Android).
- Thanh toán / Tính năng thương mại điện tử.
- Hệ thống nhắn tin trực tiếp giữa người dùng.
- GraphQL API (định hướng sau khóa học).

1.3. Định nghĩa, Từ viết tắt và Ký hiệu

Thuật ngữ / Viết tắt	Định nghĩa đầy đủ
SRS	Software Requirements Specification – Đặc tả Yêu cầu Phần mềm.
FR	Functional Requirement – Yêu cầu chức năng.
NFR	Non-Functional Requirement – Yêu cầu phi chức năng.
API	Application Programming Interface – Giao diện lập trình ứng dụng.
REST	Representational State Transfer – Kiểu kiến trúc API phổ biến nhất.
JWT	JSON Web Token – Chuẩn token xác thực stateless (RFC 7519).
RBAC	Role-Based Access Control – Kiểm soát truy cập dựa trên vai trò.
CQRS	Command Query Responsibility Segregation – Pattern tách biệt lệnh và truy vấn.
DDD	Domain-Driven Design – Phương pháp thiết kế phần mềm lấy domain làm trung tâm.
ORM	Object-Relational Mapper – Công cụ ánh xạ object-database (EF Core).
FTS	Full-Text Search – Tìm kiếm toàn văn bản.
ISR	Incremental Static Regeneration – Kỹ thuật tái tạo trang tĩnh của Next.js.

Thuật ngữ / Viết tắt	Định nghĩa đầy đủ
LCP	Largest Contentful Paint – Core Web Vital đo tốc độ tải nội dung lớn nhất.
CLS	Cumulative Layout Shift – Core Web Vital đo độ ổn định bố cục trang.
INP	Interaction to Next Paint – Core Web Vital đo thời gian phản hồi tương tác.
CI/CD	Continuous Integration / Continuous Delivery – Tích hợp và triển khai liên tục.
DXA	Device-independent pixel unit used in OOXML (1 inch = 1440 DXA).
TTL	Time-To-Live – Thời gian sống của dữ liệu trong cache.
SSR	Server-Side Rendering – Render HTML trên server.
SSG	Static Site Generation – Tạo trang tĩnh lúc build time.
MoSCoW	Must Have / Should Have / Could Have / Won't Have – Mô hình phân loại ưu tiên.
RFC	Request For Comments – Tài liệu tiêu chuẩn kỹ thuật (e.g., RFC 7807).
ERD	Entity Relationship Diagram – Sơ đồ quan hệ thực thể.
PBKDF2	Password-Based Key Derivation Function 2 – Thuật toán hash mật khẩu an toàn.
CDN	Content Delivery Network – Mạng phân phối nội dung.
MIME	Multipurpose Internet Mail Extensions – Chuẩn định dạng tệp trên Internet.
JSON-LD	JavaScript Object Notation for Linked Data – Định dạng dữ liệu có cấu trúc cho SEO.

1.4. Tài liệu Tham chiếu

STT	Tài liệu / Tiêu chuẩn	Nguồn / URL
1	IEEE Std 830-1998 – Recommended Practice for Software Requirements Specifications	https://ieeexplore.ieee.org/document/720574
2	ISO/IEC/IEEE 29148:2018 – Requirements Engineering	https://www.iso.org/standard/72089.html
3	OWASP Top 10:2021 – Top 10	https://owasp.org/www-project-top-ten/

STT	Tài liệu / Tiêu chuẩn	Nguồn / URL
	Web Application Security Risks	
4	RFC 7807 – Problem Details for HTTP APIs	https://datatracker.ietf.org/doc/html/rfc7807
5	RFC 7519 – JSON Web Token (JWT)	https://datatracker.ietf.org/doc/html/rfc7519
6	RFC 6749 – The OAuth 2.0 Authorization Framework	https://datatracker.ietf.org/doc/html/rfc6749
7	.NET 10 Minimal APIs – Microsoft Learn	https://learn.microsoft.com/aspnet/core/fundamentals/minimal-apis
8	ASP.NET Core Identity – Microsoft Learn	https://learn.microsoft.com/aspnet/core/security/authentication/identity
9	Entity Framework Core 10 Documentation	https://learn.microsoft.com/ef/core/
10	Next.js 15 App Router Documentation	https://nextjs.org/docs
11	PostgreSQL 16 Documentation – Full-Text Search	https://www.postgresql.org/docs/16/textsearch.html
12	Redis 7 Documentation	https://redis.io/docs/
13	MinIO S3-Compatible Object Storage	https://min.io/docs/
14	Google Web Vitals – Core Web Vitals	https://web.dev/explore/learn-core-web-vitals
15	Schema.org Recipe – Structured Data	https://schema.org/Recipe
16	OpenTelemetry .NET Documentation	https://opentelemetry.io/docs/languages/dotnet/
17	Serilog Documentation	https://serilog.net/
18	Hangfire Documentation	https://docs.hangfire.io/

STT	Tài liệu / Tiêu chuẩn	Nguồn / URL
19	FluentValidation Documentation	https://docs.fluentvalidation.net/
20	Giáo trình Phát triển Ứng dụng Web Nâng cao V4 – Nội bộ	N/A (tài liệu nội bộ)

1.5. Tổng quan Tài liệu

Tài liệu SRS này được tổ chức thành 8 chương chính và 3 phụ lục, theo cấu trúc từ tổng quan đến chi tiết:

- **Chương 2 – Mô tả Tổng quan:** Bối cảnh sản phẩm, chức năng tóm tắt, các lớp người dùng, môi trường vận hành và ràng buộc thiết kế.
- **Chương 3 – Yêu cầu Chức năng:** 27 FR được đặc tả chi tiết theo format chuẩn, nhóm thành 7 module chức năng.
- **Chương 4 – Yêu cầu Phi chức năng:** Hiệu năng, bảo mật, khả năng sử dụng, độ tin cậy, khả năng bảo trì/mở rộng và SEO.
- **Chương 5 – Giao diện Ngoài:** Tích hợp với các hệ thống và dịch vụ ngoài (Google OAuth, MinIO, Redis, SendGrid).
- **Chương 6 – Kiến trúc Hệ thống:** Clean Architecture Backend, Next.js App Router Frontend, chiến lược caching và deployment.
- **Chương 7 – Mô hình Dữ liệu:** ERD mô tả văn bản và bảng định nghĩa chi tiết từng entity/table.
- **Chương 8 – Đặc tả API REST:** Quy ước, chuẩn lỗi RFC 7807, và bảng tổng hợp tất cả ~30 endpoint.
- **Phụ lục A-C:** HTTP Status Codes, Application Error Codes, và Từ điển thuật ngữ.

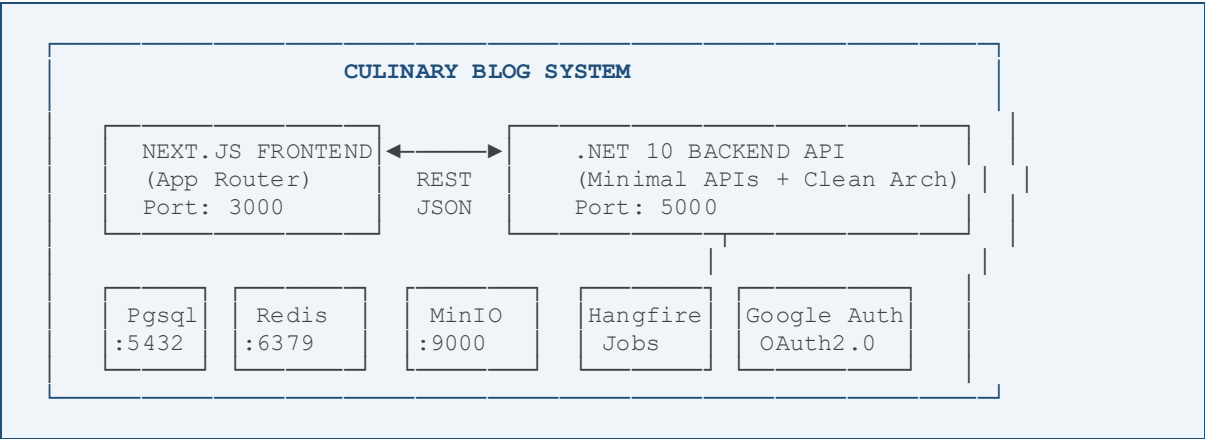
CHƯƠNG 2. MÔ TẢ TỔNG QUAN HỆ THỐNG

2.1. Bối cảnh Sản phẩm

2.1.1. Vị trí trong Hệ sinh thái

Culinary Blog vận hành theo mô hình API-Driven Architecture, trong đó Backend (.NET 10) và Frontend (Next.js) là hai hệ thống độc lập giao tiếp hoàn toàn qua HTTP/JSON RESTful API. Không có server-side rendering truyền thống (MVC Razor/Blazor) hay shared view engine giữa hai tầng.

Sơ đồ bối cảnh hệ thống (Context Diagram):



Hình 2.1. Sơ đồ bối cảnh hệ thống Culinary Blog

2.1.2. Quan hệ với Hệ thống Ngoài

Hệ thống Ngoài	Vai trò	Giao thức / Chuẩn	Hướng tích hợp
PostgreSQL 16	Hệ quản trị CSDL quan hệ chính (RDBMS)	TCP + Npgsql Driver (EF Core)	Backend → PostgreSQL
Redis 7	Distributed Cache & Session Store	TCP + StackExchange.Redis	Backend → Redis
MinIO (S3)	Object Storage cho ảnh công thức	HTTP/S3 API + MinIO .NET SDK	Backend → MinIO
Google OAuth 2.0	Đăng nhập bên thứ ba (Identity Provider)	HTTPS + OpenID Connect	Client ↔ Google ↔ Backend
Hangfire	Background Job Processing (embedded)	In-process (.NET)	Backend (internal)
Serilog / Seq	Structured Log Aggregation (development)	HTTP Sink → Seq	Backend → Seq
OpenTelemetry Collector	Distributed Tracing & Metrics (production)	OTLP / gRPC	Backend → Collector

Hệ thống Ngoài	Vai trò	Giao thức / Chuẩn	Hướng tích hợp
Nginx (Reverse Proxy)	SSL termination, load balancing, static serving	HTTP/HTTPS	Client → Nginx → Services

2.2. Chức năng Sản phẩm Tổng quát

Culinary Blog cung cấp 7 nhóm chức năng chính, được hiện thực hóa qua 27 Functional Requirements chi tiết tại Chương 3:

Nhóm chức năng	Mã nhóm	Số FR	Mô tả tóm tắt
Xác thực & Quản lý Người dùng	FR-AUTH	7	Đăng ký, đăng nhập (email + Google), JWT refresh token, logout, quản lý profile.
Quản lý Danh mục	FR-CAT	5	CRUD danh mục công thức (Category) – phân quyền Admin.
Quản lý Công thức nấu ăn	FR-RCP	10	CRUD recipe, publish/archive, quản lý ảnh/bước/nguyên liệu.
Tìm kiếm & Phân trang	FR-SRCH	4	Full-Text Search (PostgreSQL), filter, sort, offset pagination.
Quản lý Tập tin	FR-FILE	2	Upload/Delete ảnh trên MinIO S3-compatible.
Background Jobs	FR-JOB	3	Email chào mừng, thumbnail generation, sitemap XML (Hangfire).
Quan sát Hệ thống	FR-OBS	3	Health checks, structured logging, distributed tracing.

2.3. Các Lớp Người dùng và Đặc điểm

Hệ thống định nghĩa 3 loại tác nhân (Actor) với quyền hạn khác nhau:

Vai trò	Mô tả	Điều kiện	Quyền hạn chính	Ưu tiên phục vụ
Khách (Guest / Anonymous)	Người dùng chưa xác thực, truy cập ứng dụng mà không có tài khoản.	Không cần tài khoản	Xem danh sách & chi tiết recipe (Published), xem danh mục, tìm kiếm. KHÔNG được tạo/sửa/xóa.	Cao (đây là đại đa số người dùng)
Tác giả (Author)	Người dùng đã đăng ký và xác thực thành công. Được tự động gán khi đăng ký.	Có tài khoản & JWT hợp lệ	+ Tất cả quyền của Guest. + Tạo/sửa/xóa recipe CỦA MÌNH. + Upload ảnh, quản lý steps/ingredients. + Publish/Archive recipe của mình.	Cao (nhà sản xuất nội dung)
Quản trị viên (Admin)	Người quản lý hệ thống với quyền cao nhất. Được	Có tài khoản & role Admin	+ Tất cả quyền của Author. + Quản lý (CRUD) danh mục. + Sửa/xóa bất kỳ	Trung bình (số

Vai trò	Mô tả	Điều kiện	Quyền hạn chính	Ưu tiên phục vụ
	gán thủ công qua database seeding.		recipe của bất kỳ Author. + Truy cập Hangfire Dashboard. + Xem structured logs.	lượng ít)

Ghi chú về phân quyền: Hệ thống triển khai 3 tầng phân quyền. (1) Role-Based Authorization: phân biệt quyền dựa trên role (Guest/Author/Admin). (2) Resource-Based Authorization: Author chỉ sửa/xóa được recipe của chính mình (AuthorId == currentUserId). (3) Policy-Based Authorization: Policy "VerifiedAuthor" yêu cầu email đã xác nhận. Admin có quyền bypass resource ownership check.

2.4. Môi trường Vận hành

2.4.1. Môi trường Server (Production)

Thành phần	Yêu cầu tối thiểu	Khuyến nghị	Ghi chú
Hệ điều hành	Linux Ubuntu 22.04 LTS	Ubuntu 22.04 LTS / Debian 12	Docker phải được cài đặt
.NET Runtime	.NET 10.0 Runtime (aspnet)	.NET 10.0.x latest patch	Cung cấp qua Docker image mcr.microsoft.com/dotnet/aspnet:10.0
Node.js	Node.js 20 LTS (build only)	Node.js 22 LTS	Chỉ cần lúc build Next.js; production dùng standalone output
PostgreSQL	PostgreSQL 16.x	PostgreSQL 16.x	Extensions: unaccent, pg_trgm bắt buộc
Redis	Redis 7.x	Redis 7.2.x	Persistent mode với AOF
MinIO	MinIO RELEASE.2024+	MinIO latest stable	Bucket policy: public-read cho recipe images
Docker	Docker Engine 24.x	Docker Engine 27.x + Compose v2	Docker Compose cho local dev và staging
Nginx	Nginx 1.24+	Nginx 1.26+ (stable)	Reverse proxy, SSL termination
RAM	4 GB minimum	8 GB+	RAM cần tăng nếu Redis cache lớn
CPU	2 vCPU minimum	4 vCPU+	CPU-intensive: FTS indexing, image processing
Disk	20 GB SSD minimum	50 GB+ SSD	MinIO object storage tốn nhiều disk

2.4.2. Môi trường Phát triển (Development)

Thành phần	Yêu cầu
.NET 10 SDK	dotnet SDK 10.0.x (bao gồm CLI và runtime)

Thành phần	Yêu cầu
Node.js	Node.js 20+ LTS với npm 10+
Docker Desktop	Docker Desktop 4.x+ (Windows/macOS) hoặc Docker Engine (Linux) – để chạy PostgreSQL, Redis, MinIO local
IDE / Editor	Visual Studio 2022 v17.12+ / Rider 2024+ / VS Code với C# Dev Kit extension
Git	Git 2.40+ với Git LFS (nếu lưu asset lớn)
Postman / Scalar	Postman hoặc Scalar UI (tích hợp sẵn, chạy tại /scalar) để test API

2.4.3. Yêu cầu Trình duyệt Client

Trình duyệt	Phiên bản tối thiểu	Ghi chú
Google Chrome	90+	Khuyến nghị chính – tốt nhất cho Developer Tools
Mozilla Firefox	88+	Hỗ trợ đầy đủ
Microsoft Edge	90+ (Chromium)	Hỗ trợ đầy đủ (Chromium-based)
Safari	14+ (macOS 11+)	Hỗ trợ đầy đủ; Safari 13 trở xuống KHÔNG đảm bảo
Mobile Chrome (Android)	90+	Responsive design, touch-friendly
Mobile Safari (iOS)	iOS 14+	Hỗ trợ đầy đủ
Internet Explorer	Mọi phiên bản	KHÔNG hỗ trợ (EOL)

2.5. Ràng buộc Thiết kế và Hiện thực

Các ràng buộc sau đây là bắt buộc và không thể thương lượng trong suốt quá trình phát triển:

Mã ràng buộc	Loại	Mô tả ràng buộc
CONS-001	Kiến trúc	Backend PHẢI tuân thủ Clean Architecture với 4 tầng riêng biệt: Domain, Application, Infrastructure, Presentation. Tầng Domain không được phụ thuộc bất kỳ thư viện ngoài nào.
CONS-002	Pattern	CQRS với MediatR là pattern bắt buộc cho tầng Application. Mỗi use case được hiện thực dưới dạng Command hoặc Query Handler riêng biệt.
CONS-003	Ngôn ngữ / Framework	Backend: .NET 10 Minimal APIs (không dùng MVC Controllers). Frontend: Next.js App Router (không dùng Pages Router).
CONS-004	Bảo mật	Xác thực PHẢI sử dụng JWT stateless (access token 15 phút, refresh token 7 ngày). Mật khẩu PHẢI được hash với PBKDF2 qua ASP.NET Core Identity.
CONS-005	API Design	API PHẢI tuân thủ RESTful design. Phản hồi lỗi PHẢI theo RFC 7807 (application/problem+json). API versioning qua URL path (/api/v1/).

Mã ràng buộc	Loại	Mô tả ràng buộc
CONS-006	Database	PostgreSQL là DBMS duy nhất. Migrations qua EF Core Code-First. Không viết raw SQL trực tiếp (dùng LINQ hoặc Raw SQL có parameterization qua EF Core).
CONS-007	File Upload	Kích thước tệp tải lên tối đa 5 MB. Định dạng chỉ chấp nhận: image/jpeg, image/png, image/webp, image/avif. Kiểm tra MIME type (không chỉ extension).
CONS-008	Validation	Input validation PHẢI qua FluentValidation kết hợp MediatR Pipeline Behavior. Không validation trong Endpoint handler.
CONS-009	Container	Ứng dụng PHẢI được đóng gói Docker. Dockerfile multi-stage build (SDK → aspNet runtime). Docker Compose cho local development.
CONS-010	Logging	Structured logging với Serilog là bắt buộc. Mọi log entry PHẢI có CorrelationId, RequestPath, UserId (khi đã xác thực).

2.6. Giả định và Phụ thuộc

2.6.1. Giả định

- Môi trường development có kết nối Internet để pull Docker images và package NuGet/npm.
- PostgreSQL, Redis và MinIO được cung cấp qua Docker Compose trong development và dưới dạng managed service (hoặc VPS) trong production.
- Người dùng cuối có trình duyệt hiện đại và kết nối Internet đủ ổn định để load ảnh từ MinIO.
- Dữ liệu test (seed) được tạo bằng thư viện Bogus với 50 recipe mẫu và 5 tác giả mẫu.
- Email service (SendGrid hoặc SMTP) được cấu hình sẵn khi triển khai production để gửi email chào mừng.
- Giới hạn dữ liệu kỳ vọng (initial scale):** ≤ 10,000 công thức, ≤ 5,000 người dùng, ≤ 50 danh mục – phù hợp với single-server deployment.

2.6.2. Phụ thuộc Bên ngoài

Phụ thuộc	Phiên bản	Mức độ ảnh hưởng nếu không khả dụng	Kế hoạch dự phòng
Google OAuth 2.0 API	v2 (OpenID Connect)	Cao – Mất chức năng đăng nhập Google	Vẫn có đăng nhập email/password. Hiển thị thông báo "Google login tạm thời không khả dụng".
MinIO / S3	MinIO RELEASE.2024+	Cao – Không upload/xem được ảnh	Fallback về local FileSystem storage (development only). Production cần MinIO.
Redis	7.x	Trung bình – Mất cache, hiệu năng giảm	Hệ thống tiếp tục hoạt động nhưng mọi request đều query database. Cache miss graceful degradation.

Phụ thuộc	Phiên bản	Mức độ ảnh hưởng nếu không khả dụng	Kế hoạch dự phòng
PostgreSQL	16.x	Rất cao – Toàn bộ hệ thống ngừng	Backup định kỳ (pg_dump). Readiness probe sẽ fail, Nginx trả 503.
Hangfire (in-process)	v1.8+	Thấp – Background jobs không chạy	Fire-and-forget jobs sẽ bị mất; Recurring jobs bỏ qua chu kỳ. Không ảnh hưởng core functionality.

CHƯƠNG 3. YÊU CẦU CHỨC NĂNG CHI TIẾT

Chương này đặc tả chi tiết 27 Functional Requirements (FR) được nhóm thành 7 module chức năng. Mỗi FR được mô tả theo template chuẩn bao gồm: Mã yêu cầu, Tên, Nhóm chức năng, Tác nhân, Mức ưu tiên (MoSCoW), Mô tả, Điều kiện tiên quyết, Luồng chính, Luồng thay thế/Ngoại lệ, HTTP Endpoint, Kết quả mong đợi và HTTP Status Code.

Quy ước mức ưu tiên MoSCoW: M (Must Have – Bắt buộc), S (Should Have – Nên có), C (Could Have – Có thể có), W (Won't Have – Không trong scope hiện tại).

3.1. Module Xác thực và Quản lý Người dùng (FR-AUTH)

Module này quản lý toàn bộ vòng đời xác thực người dùng: từ đăng ký, đăng nhập đa phương thức, duy trì phiên làm việc với cơ chế token rotation, đến quản lý hồ sơ cá nhân. Backend sử dụng ASP.NET Core Identity kết hợp JWT và OAuth 2.0.

FR-AUTH-001: Đăng ký Tài khoản (User Registration)

Mã yêu cầu	FR-AUTH-001
Tên yêu cầu	Đăng ký Tài khoản Mới
Nhóm chức năng	Module Xác thực và Quản lý Người dùng (FR-AUTH)
Tác nhân	Khách (Guest / Anonymous User)
Mức ưu tiên (MoSCoW)	M – Must Have (Bắt buộc)
Mô tả	Hệ thống cho phép người dùng chưa có tài khoản tạo một tài khoản mới bằng cách cung cấp thông tin cơ bản. Sau khi đăng ký thành công, người dùng tự động được gán role "Author" và nhận bộ token để truy cập ngay lập tức (auto-login sau đăng ký). Hệ thống kích hoạt job gửi email chào mừng bất đồng bộ qua Hangfire.
Điều kiện tiên quyết	1. Người dùng chưa đăng nhập vào hệ thống. 2. Endpoint POST /api/v1/auth/register đang hoạt động. 3. PostgreSQL database đang kết nối thành công.
Luồng chính (Happy Path)	- Người dùng (client) gửi HTTP POST đến /api/v1/auth/register với JSON body: { "fullName": "...", "email": "...", "userName": "...", "password": "..." }. - RegisterCommand được tạo và dispatch đến MediatR. - ValidationBehavior chạy RegisterCommandValidator: kiểm tra fullName không rỗng, email đúng format, userName không chứa ký tự đặc biệt, password tối thiểu 8 ký tự (1 chữ hoa, 1 chữ số, 1 ký tự đặc biệt). - RegisterCommandHandler kiểm tra email chưa tồn tại trong database (userManager.FindByEmailAsync). - Tạo ApplicationUser mới qua factory method ApplicationUser.Create(fullName, email, userName). - userManager.CreateAsync(user, password) – ASP.NET Core Identity tự hash password với PBKDF2.

	7. UserManager.AddToRoleAsync(user, "Author") – gán role mặc định. 8. JwtService.GenerateAccessToken() – tạo JWT access token (HS256, 15 phút). 9. JwtService.GenerateRefreshToken() – tạo refresh token ngẫu nhiên (512-bit, 7 ngày). 10. Lưu RefreshToken vào bảng refresh_tokens trong database. 11. BackgroundJob.Enqueue<WelcomeEmailJob>() – đẩy job gửi email chào mừng vào Hangfire queue (fire-and-forget). 12. Trả về HTTP 201 Created với AuthResponseDto: { accessToken, refreshToken, expiresAt, user: { id, fullName, email, userName, avatarUrl, roles } }.
Luồng thay thế / Ngoại lệ	A1 – Email đã tồn tại: Tại bước 4, nếu email đã được đăng ký → Throw ConflictException → GlobalExceptionHandlerMiddleware trả về HTTP 409 Conflict với RFC 7807 body. A2 – Password không đủ mạnh: Tại bước 3 hoặc 6, UserManager.CreateAsync trả về IdentityError → Throw ValidationException → HTTP 422 Unprocessable Entity với danh sách lỗi chi tiết. A3 – Dữ liệu đầu vào không hợp lệ: Tại bước 3, FluentValidation fail → HTTP 422 với từng field lỗi (theo RFC 7807 ValidationProblemDetails). A4 – Database không kết nối: EF Core ném DbUpdateException → HTTP 500 Internal Server Error (GlobalExceptionHandlerMiddleware log lỗi, không expose stack trace).
HTTP Method & Endpoint	POST /api/v1/auth/register
Kết quả mong đợi	Tài khoản mới được tạo trong database, role "Author" được gán, refresh token được persist, email chào mừng được đẩy vào Hangfire queue. Client nhận được access token và refresh token.
HTTP Status Code trả về	201 Created – Đăng ký thành công. 409 Conflict – Email đã tồn tại. 422 Unprocessable Entity – Dữ liệu không hợp lệ. 500 Internal Server Error – Lỗi hệ thống.

FR-AUTH-002: Đăng nhập bằng Email/Mật khẩu (Local Login)

Mã yêu cầu	FR-AUTH-002
Tên yêu cầu	Đăng nhập bằng Email và Mật khẩu
Nhóm chức năng	Module Xác thực và Quản lý Người dùng (FR-AUTH)
Tác nhân	Tác giả đã đăng ký (Author) hoặc Quản trị viên (Admin)
Mức ưu tiên (MoSCoW)	M – Must Have (Bắt buộc)
Mô tả	Hệ thống cho phép người dùng đã có tài khoản đăng nhập bằng email và mật khẩu. Mỗi lần đăng nhập thành công tạo ra một cặp access token mới (JWT, 15 phút) và refresh token mới (7 ngày). Cơ chế Token Rotation: refresh token cũ KHÔNG bị xóa ngay mà được đánh dấu đã sử dụng (để phát hiện token reuse attack).
Điều kiện tiên quyết	1. Người dùng đã có tài khoản hợp lệ trong hệ thống. 2. Tài khoản chưa bị khóa (LockoutEnabled = false hoặc chưa đến lockout deadline).

Luồng chính (Happy Path)	- Client gửi POST /api/v1/auth/login với body: { "email": "...", "password": "..." }. - LoginCommand được dispatch qua MediatR. - ValidationBehavior kiểm tra email format và password không rỗng. - LoginCommandHandler tìm user: UserManager.FindByEmailAsync(email). - Xác minh mật khẩu: UserManager.CheckPasswordAsync(user, password) – so sánh với PBKDF2 hash. - Kiểm tra tài khoản không bị logout: UserManager.IsLockedOutAsync(user). - Tạo access token mới: JwtService.GenerateAccessToken(user, roles). - Tạo refresh token mới: JwtService.GenerateRefreshToken(userId). - Lưu refresh token mới vào database. - Ghi nhận đăng nhập thành công: UserManager.ResetAccessFailedCountAsync(user). - Trả về HTTP 200 OK với AuthResponseDto.
Luồng thay thế / Ngoại lệ	A1 – Tài khoản không tồn tại hoặc mật khẩu sai: HTTP 401 Unauthorized với message generic "Email hoặc mật khẩu không đúng" (KHÔNG tiết lộ tài khoản có tồn tại hay không – tránh User Enumeration Attack). A2 – Tài khoản bị logout: HTTP 423 Locked với thông báo thời gian unlock còn lại. A3 – Vượt quá số lần thử sai (5 lần): AccessFailedCount tăng lên, sau 5 lần → tài khoản bị logout 15 phút (cấu hình qua LockoutOptions).
HTTP Method & Endpoint	POST /api/v1/auth/login
Kết quả mong đợi	Access token và refresh token mới được tạo và trả về. Refresh token được lưu vào database.
HTTP Status Code trả về	200 OK – Đăng nhập thành công. 401 Unauthorized – Sai email/mật khẩu. 422 Unprocessable Entity – Dữ liệu không hợp lệ. 423 Locked – Tài khoản bị khóa.

FR-AUTH-003: Đăng nhập bằng Google OAuth 2.0

Mã yêu cầu	FR-AUTH-003
Tên yêu cầu	Đăng nhập / Đăng ký bằng Google OAuth 2.0
Nhóm chức năng	Module Xác thực và Quản lý Người dùng (FR-AUTH)
Tác nhân	Khách (Guest) – lần đầu / Người dùng đã đăng ký trước đó qua Google
Mức ưu tiên (MoSCoW)	S – Should Have
Mô tả	Hệ thống hỗ trợ đăng nhập qua tài khoản Google sử dụng OAuth 2.0 Authorization Code Flow với PKCE. Nếu đây là lần đăng nhập Google đầu tiên, hệ thống tự động tạo tài khoản mới từ thông tin Google profile (email, display name, avatar URL) và gán role "Author". Nếu email đã tồn tại từ đăng ký thủ công trước đó, hệ thống liên kết Google login với tài khoản hiện có.

Điều kiện tiên quyết	1. Google OAuth 2.0 Credentials (ClientId, ClientSecret) đã được cấu hình trong appsettings. 2. Redirect URI đã được đăng ký trong Google Cloud Console. 3. Người dùng có tài khoản Google hợp lệ.
Luồng chính (Happy Path)	1. Frontend (Next.js) redirect người dùng đến Google Authorization Endpoint với scopes: openid, email, profile. 2. Người dùng xác nhận cấp quyền trên Google Consent Screen. 3. Google redirect về callback URL (Next.js) với Authorization Code. 4. Auth.js v5 (Next.js) xử lý callback, lấy access token từ Google và lấy profile. 5. Frontend gửi POST /api/v1/auth/google với Google ExternalLoginInfo. 6. GoogleLoginCommandHandler tìm user bằng UserManager.FindByLoginAsync("Google", providerKey). 7. Nếu chưa có tài khoản: kiểm tra email → nếu email chưa tồn tại thì tạo ApplicationUser mới từ Google profile, gán role "Author" → AddLoginAsync. 8. Nếu email đã tồn tại (đã đăng ký thủ công): liên kết Google login → AddLoginAsync với tài khoản hiện có. 9. Tạo access token và refresh token, lưu vào database. 10. Trả về HTTP 200 OK với AuthResponseDto.
Luồng thay thế / Ngoại lệ	A1 – Google token không hợp lệ hoặc hết hạn: HTTP 401 Unauthorized. A2 – Email Google bị revoke quyền: HTTP 400 Bad Request. A3 – Google API không khả dụng: HTTP 502 Bad Gateway với message thích hợp.
HTTP Method & Endpoint	POST /api/v1/auth/google
Kết quả mong đợi	Người dùng được đăng nhập (hoặc tự động đăng ký), nhận AuthResponseDto. Tài khoản mới (nếu có) được tạo với role "Author".
HTTP Status Code trả về	200 OK – Đăng nhập/đăng ký thành công. 401 Unauthorized – Token Google không hợp lệ. 400 Bad Request – Thiếu thông tin Google profile.

FR-AUTH-004: Làm mới Access Token (Token Refresh)

Mã yêu cầu	FR-AUTH-004
Tên yêu cầu	Làm mới Access Token bằng Refresh Token
Nhóm chức năng	Module Xác thực và Quản lý Người dùng (FR-AUTH)
Tác nhân	Tác giả (Author) / Quản trị viên (Admin) – có refresh token hợp lệ
Mức ưu tiên (MoSCoW)	M – Must Have (Bắt buộc)
Mô tả	Khi access token hết hạn (sau 15 phút), client sử dụng refresh token còn hiệu lực để lấy cặp token mới mà không cần người dùng đăng nhập lại. Cơ chế Token Rotation bắt buộc: mỗi lần refresh, refresh token cũ bị vô hiệu hóa (IsRevoked = true, RevokedAt = DateTime.UtcNow) và một refresh token MỚI được tạo ra. Đây là biện pháp chống Refresh Token Reuse Attack.

Điều kiện tiên quyết	1. Client có refresh token hợp lệ (chưa hết hạn, chưa bị revoke, chưa bị thay thế). 2. Người dùng tương ứng vẫn còn tồn tại trong database và chưa bị khóa.
Luồng chính (Happy Path)	- Client gửi POST /api/v1/auth/refresh với body: { "refreshToken": "..." }. - RefreshTokenCommand dispatch qua MediatR. - Handler tìm refresh token trong database: bao gồm User navigation property. - Kiểm tra: token tồn tại, IsRevoked == false, ExpiresAt > DateTime.UtcNow, user vẫn active. - Đánh dấu token cũ: IsRevoked = true, ReplacedByToken = newToken, RevokedAt = DateTime.UtcNow. - Tạo access token mới cho user. - Tạo refresh token mới, lưu vào database. - Trả về HTTP 200 OK với AuthResponseDto chứa cặp token mới.
Luồng thay thế / Ngoại lệ	A1 – Refresh token không tìm thấy trong database: HTTP 401 Unauthorized. A2 – Refresh token đã hết hạn: HTTP 401 Unauthorized, client phải đăng nhập lại. A3 – Refresh token đã bị revoke (Reuse Attack detected): HTTP 401 Unauthorized. LOG SECURITY ALERT với mức WARNING. Có thể kích hoạt revoke toàn bộ refresh tokens của user đó (paranoid mode). A4 – User bị xóa hoặc bị khóa sau khi token được cấp: HTTP 401 Unauthorized.
HTTP Method & Endpoint	POST /api/v1/auth/refresh
Kết quả mong đợi	Refresh token cũ bị invalidate. Access token mới (15 phút) và refresh token mới (7 ngày) được tạo và trả về.
HTTP Status Code trả về	200 OK – Refresh thành công. 401 Unauthorized – Token không hợp lệ, hết hạn hoặc đã bị revoke.

FR-AUTH-005: Đăng xuất (Logout / Token Revocation)

Mã yêu cầu	FR-AUTH-005
Tên yêu cầu	Đăng xuất và Thu hồi Refresh Token
Nhóm chức năng	Module Xác thực và Quản lý Người dùng (FR-AUTH)
Tác nhân	Tác giả (Author) / Quản trị viên (Admin) đang đăng nhập
Mức ưu tiên (MoSCoW)	M – Must Have
Mô tả	Người dùng đăng xuất khỏi hệ thống. Vì JWT access token là stateless (không thể revoke trực tiếp trước khi hết hạn), hành động logout chủ yếu là revoke refresh token tương ứng trong database. Client có trách nhiệm xóa access token khỏi bộ nhớ (localStorage/cookie) phía client.
Điều kiện tiên quyết	1. Người dùng đang đăng nhập với access token hợp lệ trong Authorization header. 2. Client gửi refresh token muốn revoke.
Luồng chính (Happy Path)	1. Client gửi POST /api/v1/auth/logout với Authorization: Bearer {accessToken} header và body: { "refreshToken": "..." }.

	2. Middleware xác thực JWT (UseAuthentication) xác minh access token. 3. LogoutCommandHandler tìm refresh token trong database. 4. Nếu tìm thấy và thuộc về user hiện tại: đánh dấu IsRevoked = true, RevokedAt = DateTime.UtcNow. 5. Lưu thay đổi vào database. 6. Trả về HTTP 204 No Content.
Luồng thay thế / Ngoại lệ	A1 – Refresh token không tìm thấy: Vẫn trả về HTTP 204 (idempotent – không tiết lộ trạng thái). A2 – Access token đã hết hạn: Vẫn cho phép logout nếu refresh token hợp lệ; hoặc HTTP 401 nếu không cung cấp refresh token.
HTTP Method & Endpoint	POST /api/v1/auth/logout
Kết quả mong đợi	Refresh token bị đánh dấu IsRevoked = true trong database. Các lần refresh tiếp theo với token này sẽ thất bại.
HTTP Status Code trả về	204 No Content – Đăng xuất thành công (hoặc token không tồn tại – idempotent). 401 Unauthorized – Access token không hợp lệ.

FR-AUTH-006: Xem Hồ sơ Cá nhân (View Profile)

Mã yêu cầu	FR-AUTH-006
Tên yêu cầu	Xem Hồ sơ Cá nhân
Nhóm chức năng	Module Xác thực và Quản lý Người dùng (FR-AUTH)
Tác nhân	Tác giả (Author) / Quản trị viên (Admin) đang đăng nhập
Mức ưu tiên (MoSCoW)	S – Should Have
Mô tả	Trả về thông tin hồ sơ của người dùng hiện đang đăng nhập, dựa trên UserId được trích xuất từ JWT claims. Không bao giờ trả về PasswordHash hoặc SecurityStamp.
Điều kiện tiên quyết	1. Người dùng đang đăng nhập với access token hợp lệ.
Luồng chính (Happy Path)	1. Client gửi GET /api/v1/auth/me với Authorization: Bearer {accessToken}. 2. Middleware xác thực JWT, trích xuất UserId từ claim NamelfIdentifier. 3. GetCurrentUserQuery dispatch qua MediatR. 4. Handler tìm user: UserManager.FindByIdAsync(userId). 5. Map sang UserProfileDto: { id, fullName, email, userName, avatarUrl, roles, emailConfirmed, createdAt }. 6. Trả về HTTP 200 OK với UserProfileDto.
Luồng thay thế / Ngoại lệ	A1 – User đã bị xóa khỏi database sau khi token được cấp: HTTP 404 Not Found.
HTTP Method & Endpoint	GET /api/v1/auth/me
Kết quả mong đợi	Trả về thông tin hồ sơ đầy đủ của người dùng (không có thông tin nhạy cảm như password hash).

HTTP Status Code trả về	200 OK – Thành công. 401 Unauthorized – Chưa đăng nhập. 404 Not Found – User không tồn tại.
--------------------------------	---

FR-AUTH-007: Cập nhật Hồ sơ Cá nhân (Update Profile)

Mã yêu cầu	FR-AUTH-007
Tên yêu cầu	Cập nhật Hồ sơ Cá nhân
Nhóm chức năng	Module Xác thực và Quản lý Người dùng (FR-AUTH)
Tác nhân	Tác giả (Author) / Quản trị viên (Admin) đang đăng nhập
Mức ưu tiên (MoSCoW)	S – Should Have
Mô tả	Người dùng có thể cập nhật FullName và AvatarUrl của mình. Email và Username không thể thay đổi qua endpoint này (đây là quy trình riêng có xác nhận OTP). Sử dụng PATCH (partial update) để chỉ cập nhật các field được cung cấp.
Điều kiện tiên quyết	1. Người dùng đang đăng nhập. 2. Dữ liệu mới phải hợp lệ (FullName không rỗng, AvatarUrl là URL hợp lệ nếu cung cấp).
Luồng chính (Happy Path)	1. Client gửi PATCH /api/v1/auth/me với body: { "fullName": "...", "avatarUrl": "..." }. 2. UpdateProfileCommand dispatch qua MediatR, UserId lấy từ JWT claims. 3. ValidationBehavior kiểm tra: fullName 2–100 ký tự, avatarUrl là URL hợp lệ (nếu cung cấp). 4. Handler tìm user, cập nhật FullName và/hoặc AvatarUrl. 5. UserManager.UpdateAsync(user). 6. Trả về HTTP 200 OK với UserProfileDto đã cập nhật.
Luồng thay thế / Ngoại lệ	A1 – Dữ liệu không hợp lệ: HTTP 422 Unprocessable Entity.
HTTP Method & Endpoint	PATCH /api/v1/auth/me
Kết quả mong đợi	Hồ sơ người dùng được cập nhật trong database. Trả về hồ sơ mới.
HTTP Status Code trả về	200 OK – Cập nhật thành công. 401 Unauthorized – Chưa đăng nhập. 422 Unprocessable Entity – Dữ liệu không hợp lệ.

3.2. Module Quản lý Danh mục (FR-CAT)

Module quản lý danh mục (Category) phân loại công thức nấu ăn. Danh mục được tạo và duy trì bởi Admin; Author và Guest chỉ có quyền đọc. Mỗi danh mục có Slug duy nhất phục vụ URL thân thiện SEO. Danh mục được cache với IMemoryCache (TTL 1 giờ) vì thay đổi ít thường xuyên.

FR-CAT-001: Xem Danh sách Danh mục

Mã yêu cầu	FR-CAT-001
Tên yêu cầu	Xem Danh sách Tất cả Danh mục

Nhóm chức năng	Module Quản lý Danh mục (FR-CAT)
Tác nhân	Tất cả (Guest / Author / Admin)
Mức ưu tiên (MoSCoW)	M – Must Have
Mô tả	Trả về danh sách tất cả danh mục công thức hiện có trong hệ thống, kèm số lượng công thức đã xuất bản (Published) trong mỗi danh mục. Kết quả được cache với IMemoryCache (TTL 60 phút) và sắp xếp theo Name tăng dần.
Điều kiện tiên quyết	1. Ít nhất một danh mục tồn tại trong database (hoặc trả về mảng rỗng). 2. Không yêu cầu xác thực.
Luồng chính (Happy Path)	1. Client gửi GET /api/v1/categories. 2. GetCategoriesQuery dispatch qua MediatR. 3. Handler kiểm tra IMemoryCache với key "categories:all". 4. Cache hit: trả về dữ liệu từ cache. 5. Cache miss: query database (UnitOfWork.Categories.GetAllWithRecipeCount()), map sang CategoryDto[]. 6. Lưu vào IMemoryCache với TTL 60 phút (sliding expiration). 7. Trả về HTTP 200 OK với CategoryDto[].
Luồng thay thế / Ngoại lệ	A1 – Không có danh mục nào: HTTP 200 OK với mảng rỗng [].
HTTP Method & Endpoint	GET /api/v1/categories
Kết quả mong đợi	Mảng CategoryDto[] với các field: { id, name, slug, description, recipeCount }. Kết quả được serve từ cache khi có.
HTTP Status Code trả về	200 OK – Thành công (kể cả khi trống).

FR-CAT-002: Xem Chi tiết Danh mục và Công thức

Mã yêu cầu	FR-CAT-002
Tên yêu cầu	Xem Chi tiết Danh mục và Danh sách Công thức thuộc Danh mục
Nhóm chức năng	Module Quản lý Danh mục (FR-CAT)
Tác nhân	Tất cả (Guest / Author / Admin)
Mức ưu tiên (MoSCoW)	M – Must Have
Mô tả	Trả về thông tin chi tiết của một danh mục cụ thể (theo Slug) kèm danh sách phân trang các công thức đã xuất bản (Published) thuộc danh mục đó. Guest chỉ thấy Published recipes; Author thấy thêm Draft recipes của chính mình trong danh mục.
Điều kiện tiên quyết	1. Danh mục với slug tương ứng phải tồn tại. 2. Không yêu cầu xác thực.
Luồng chính (Happy Path)	1. Client gửi GET /api/v1/categories/{slug}?page=1&pageSize=12. 2. GetCategoryBySlugQuery dispatch qua MediatR.

	3. Handler tìm category theo slug: <code>_unitOfWork.Categories.GetBySlugAsync(slug)</code> . 4. Query recipes thuộc category với Status == Published (+ Draft của currentUser nếu đã đăng nhập). 5. Apply pagination (OFFSET-based: <code>SKIP (page-1)*pageSize TAKE pageSize</code>). 6. Map sang <code>CategoryDetailDto</code> kèm <code>PagedResult<RecipeSummaryDto></code> . 7. Trả về HTTP 200 OK.
Luồng thay thế / Ngoại lệ	A1 – Slug không tồn tại: HTTP 404 Not Found với RFC 7807 body.
HTTP Method & Endpoint	GET <code>/api/v1/categories/{slug}?page={n}&pageSize={n}</code>
Kết quả mong đợi	{ category: CategoryDto, recipes: { items: RecipeSummaryDto[], totalCount, page, pageSize, totalPages } }
HTTP Status Code trả về	200 OK – Thành công. 404 Not Found – Slug không tồn tại.

FR-CAT-003: Tạo Danh mục Mới [Admin]

Mã yêu cầu	FR-CAT-003
Tên yêu cầu	Tạo Danh mục Công thức Mới
Nhóm chức năng	Module Quản lý Danh mục (FR-CAT)
Tác nhân	Quản trị viên (Admin)
Mức ưu tiên (MoSCoW)	M – Must Have
Mô tả	Admin tạo danh mục công thức mới. Slug được tự động sinh từ Name (slugify: chuyển sang chữ thường, bỏ dấu, thay khoảng trắng bằng "-"). Nếu Slug đã tồn tại, hệ thống thêm suffix số (e.g., "mon-chinh-2"). Sau khi tạo, cache danh mục (IMemoryCache key "categories:all") bị invalidate.
Điều kiện tiên quyết	1. Người dùng đang đăng nhập với role Admin. 2. Name chưa tồn tại trong database.
Luồng chính (Happy Path)	1. Admin gửi POST <code>/api/v1/categories</code> với Authorization: Bearer {adminJwt} và body: { "name": "...", "description": "..." }. 2. <code>RequireAuthorization("Admin")</code> middleware kiểm tra role. 3. <code>CreateCategoryCommand</code> dispatch qua MediatR. 4. <code>ValidationBehavior</code> : name 2–50 ký tự, không chứa HTML. 5. <code>SlugHelper.Generate(name)</code> tạo slug. 6. Kiểm tra slug chưa tồn tại. Nếu trùng, thêm "-2", "-3",... cho đến khi unique. 7. <code>Category.Create(name, slug, description)</code> tạo entity. 8. <code>_unitOfWork.Categories.AddAsync(entity)</code> . 9. <code>_unitOfWork.SaveChangesAsync()</code> . 10. <code>MemoryCache.Remove("categories:all")</code> – invalidate cache. 11. Trả về HTTP 201 Created với <code>CategoryDto</code> và Location header.

Luồng thay thế / Ngoại lệ	A1 – Thiếu role Admin: HTTP 403 Forbidden. A2 – Dữ liệu không hợp lệ: HTTP 422.
HTTP Method & Endpoint	POST /api/v1/categories
Kết quả mong đợi	Danh mục mới được tạo trong database. Cache danh mục bị xóa. Location header trở đến /api/v1/categories/{newSlug}.
HTTP Status Code trả về	201 Created – Tạo thành công. 403 Forbidden – Không có quyền Admin. 409 Conflict – Name đã tồn tại. 422 Unprocessable Entity – Dữ liệu không hợp lệ.

FR-CAT-004: Cập nhật Danh mục [Admin]

Mã yêu cầu	FR-CAT-004
Tên yêu cầu	Cập nhật Thông tin Danh mục
Nhóm chức năng	Module Quản lý Danh mục (FR-CAT)
Tác nhân	Quản trị viên (Admin)
Mức ưu tiên (MoSCoW)	M – Must Have
Mô tả	Admin cập nhật Name và/hoặc Description của danh mục. Slug KHÔNG thay đổi khi đổi tên (để tránh broken links). Sau khi cập nhật, cache bị invalidate.
Điều kiện tiên quyết	1. Admin đang đăng nhập. 2. Danh mục với ID tương ứng tồn tại.
Luồng chính (Happy Path)	1. Admin gửi PUT /api/v1/categories/{id} với body: { "name": "...", "description": "..." }. 2. Kiểm tra role Admin. 3. UpdateCategoryCommand dispatch qua MediatR. 4. Tìm category theo ID, cập nhật Name và Description. 5. Lưu thay đổi, invalidate cache. 6. Trả về HTTP 200 OK với CategoryDto đã cập nhật.
Luồng thay thế / Ngoại lệ	A1 – ID không tồn tại: HTTP 404. A2 – Thiếu role Admin: HTTP 403.
HTTP Method & Endpoint	PUT /api/v1/categories/{id:guid}
Kết quả mong đợi	Thông tin danh mục được cập nhật. Cache invalidated.
HTTP Status Code trả về	200 OK – Cập nhật thành công. 403 Forbidden. 404 Not Found. 422 Unprocessable Entity.

FR-CAT-005: Xóa Danh mục [Admin]

Mã yêu cầu	FR-CAT-005
Tên yêu cầu	Xóa Danh mục
Nhóm chức năng	Module Quản lý Danh mục (FR-CAT)
Tác nhân	Quản trị viên (Admin)

Mức ưu tiên (MoSCoW)	S – Should Have
Mô tả	Admin xóa một danh mục. Quy tắc nghiệp vụ: KHÔNG được xóa danh mục còn chứa công thức (dù là Published hay Draft). Admin phải chuyển tất cả công thức sang danh mục khác trước khi xóa. Đây là soft constraint để bảo vệ toàn vẹn dữ liệu.
Điều kiện tiên quyết	1. Admin đang đăng nhập. 2. Danh mục tồn tại và không còn công thức nào.
Luồng chính (Happy Path)	1. Admin gửi DELETE /api/v1/categories/{id}. 2. Kiểm tra role Admin. 3. DeleteCategoryCommand dispatch. 4. Đếm số recipe trong category: nếu > 0 → Throw ConflictException("Danh mục còn chứa {count} công thức."). 5. Xóa entity, lưu thay đổi, invalidate cache. 6. Trả về HTTP 204 No Content.
Luồng thay thế / Ngoại lệ	A1 – Danh mục có recipe: HTTP 409 Conflict với thông báo số lượng recipe. A2 – ID không tồn tại: HTTP 404.
HTTP Method & Endpoint	DELETE /api/v1/categories/{id:guid}
Kết quả mong đợi	Danh mục bị xóa khỏi database. HTTP 204 được trả về.
HTTP Status Code trả về	204 No Content – Xóa thành công. 403 Forbidden. 404 Not Found. 409 Conflict – Danh mục còn recipe.

3.3. Module Quản lý Công thức Nấu ăn (FR-RCP)

Module cốt lõi của hệ thống. Recipe là aggregate root chứa các child entity: RecipeStep, RecipeIngredient, RecipeImage và Owned Entity RecipeNutrition. Tất cả mutation (Create/Update/Delete) đi qua UnitOfWork để đảm bảo tính nhất quán transaction. Concurrency được xử lý qua RowVersion (Timestamp) để phát hiện lost update khi hai Author cùng sửa một recipe.

FR-RCP-001: Xem Danh sách Công thức (Paginated + Filtered + Sorted)

Mã yêu cầu	FR-RCP-001
Tên yêu cầu	Xem Danh sách Công thức Nấu ăn với Phân trang, Lọc và Sắp xếp
Nhóm chức năng	Module Quản lý Công thức Nấu ăn (FR-RCP)
Tác nhân	Tất cả (Guest / Author / Admin)
Mức ưu tiên (MoSCoW)	M – Must Have

Mô tả	Trả về danh sách phân trang các công thức. Guest và Author khác chỉ thấy Status == Published. Author thêm Draft/Archived của chính mình. Admin thấy tất cả trạng thái. Hỗ trợ lọc theo CategoryId, Difficulty, thời gian nấu; sắp xếp theo createdAt, title, cookTime. Kết quả được cache với Output Cache (.NET 1 policy "RecipeList" (TTL 15 phút, vary by query string).
Điều kiện tiên quyết	1. Không yêu cầu xác thực (endpoint public cho Published recipes). 2. Tham số page >= 1, pageSize <= 50].
Luồng chính (Happy Path)	1. Client gửi GET /api/v1/recipes?page=1&pageSize=12&categoryId={guid}&difficulty=Easy&maxCookTime=30&sort=-createdAt 2. GetRecipesQuery dispatch qua MediatR. 3. Handler xây dựng IQueryable với filters từ query params. 4. Áp dụng Authorization filter: nếu Guest → chỉ Published; nếu Author → Published OR (Draft AND AuthorId == userId); nếu Admin → tất cả. 5. Apply sorting: sort="-createdAt" → ORDER BY CreatedAt DESC; sort="title" → ORDER BY Title ASC 6. COUNT total trước khi pagination. 7. Apply OFFSET-LIMIT pagination. 8. Map sang PagedResult<RecipeSummaryDto>. 9. Trả về HTTP 200 OK. Output Cache lưu response theo key = {path}?{queryString}.
Luồng thay thế / Ngoại lệ	A1 – page hoặc pageSize không hợp lệ: HTTP 422. A2 – categoryId không tồn tại: HTTP 200 với itemNotFound (không throw 404).
HTTP Method & Endpoint	GET /api/v1/recipes?page={n}&pageSize={n}&categoryId={guid}&difficulty={level}&maxCookTime={min}&sort={order}
Kết quả mong đợi	PagedResult<RecipeSummaryDto>: { items[], totalCount, page, pageSize, totalPages, hasNextPage, hasPreviousPage }.
HTTP Status Code trả về	200 OK – Thành công (kể cả items rỗng). 422 Unprocessable Entity – Tham số không hợp lệ.

FR-RCP-002: Xem Chi tiết Công thức

Mã yêu cầu	FR-RCP-002
Tên yêu cầu	Xem Chi tiết Công thức Nấu ăn
Nhóm chức năng	Module Quản lý Công thức Nấu ăn (FR-RCP)
Tác nhân	Tất cả (Guest / Author / Admin)
Mức ưu tiên (MoSCoW)	M – Must Have
Mô tả	Trả về toàn bộ thông tin chi tiết của một công thức cụ thể, bao gồm: thông tin cơ bản, danh sách nguyên liệu (RecipeIngredient[]) sắp xếp theo SortOrder, các bước thực hiện (RecipeStep[]) sắp xếp theo StepNumber, hình ảnh minh họa (RecipeImage[]), thông tin dinh dưỡng (RecipeNutrition), thông tin danh mục và tác giả. Recipe chỉ được xem bởi tác giả sở hữu hoặc Admin. Endpoint được cache với Output Cache policy "RecipeDetail" (TTL 60 phút) và tagged với "recipes" để hỗ trợ tag-based invalidation.

Điều kiện tiên quyết	1. Recipe với slug tương ứng tồn tại. 2. Nếu Recipe ở trạng thái Draft/Archived: người yêu cầu phải là người dùng hoặc Admin.
Luồng chính (Happy Path)	- Client gửi GET /api/v1/recipes/{slug}. - GetRecipeBySlugQuery dispatch qua MediatR. - Handler query Recipe với Eager Loading: Include(Steps).Include(Ingredients).Include/Images).Include(Category).Include(Author).IncludeOwned(Ingredients). - Kiểm tra null → NotFoundException nếu không tìm thấy. - Kiểm tra Status: nếu Draft/Archived → chỉ tác giả hoặc Admin mới được xem (Authorization check). - Map sang RecipeDetailDto (bao gồm tất cả nested collections). - Trả về HTTP 200 OK. Tag output cache entry với ["recipes", \$"recipe:{slug}"].
Luồng thay thế / Ngoại lệ	A1 – Slug không tồn tại: HTTP 404 Not Found. A2 – Recipe Draft/Archived, người dùng không có quyền: HTTP 403 Forbidden.
HTTP Method & Endpoint	GET /api/v1/recipes/{slug}
Kết quả mong đợi	RecipeDetailDto đầy đủ gồm tất cả nested data (steps, ingredients, images, nutrition, category, author).
HTTP Status Code trả về	200 OK – Thành công. 403 Forbidden – Không có quyền xem Draft. 404 Not Found – Slug không tồn tại.

FR-RCP-003: Tạo Công thức Nấu ăn Mới [Author/Admin]

Mã yêu cầu	FR-RCP-003
Tên yêu cầu	Tạo Công thức Nấu ăn Mới
Nhóm chức năng	Module Quản lý Công thức Nấu ăn (FR-RCP)
Tác nhân	Tác giả (Author) / Quản trị viên (Admin)
Mức ưu tiên (MoSCoW)	M – Must Have
Mô tả	Author hoặc Admin tạo mới một công thức nấu ăn. Trạng thái ban đầu luôn là Draft (chưa công khai). Slug được tự động sinh từ Title. Steps và Ingredients có thể được tạo cùng lúc (trong cùng request) hoặc thêm riêng lẻ sau qua FR-RCP-009/010.
Điều kiện tiên quyết	1. Người dùng đang đăng nhập với role Author hoặc Admin. 2. CategoryId tham chiếu đến danh mục đã tồn tại.
Luồng chính (Happy Path)	- Author gửi POST /api/v1/recipes với body: { title, description, categoryId, prepTimeMinutes, cookTimeMinutes, servings, difficulty, instructions?, nutrition?: {...}, steps?: [...], ingredients?: [...] }. - Kiểm tra xác thực (RequireAuthorization). - CreateRecipeCommand dispatch. - ValidationBehavior: title 5–200 ký tự, prepTime/cookTime/servings > 0, categoryId valid Guid. - SlugHelper.Generate(title), kiểm tra slug unique. - Recipe.Create(title, description, categoryId, authorId, prepTime, cookTime, servings, difficulty).

	7. Nếu có steps: thêm từng RecipeStep.Create() vào recipe.Steps. 8. Nếu có ingredients: thêm từng RecipeIngredient.Create() vào recipe.Ingredients. 9. Nếu có nutrition: recipe.SetNutrition(calories, protein, carbs, fat). 10. _unitOfWork.Recipes.AddAsync(recipe), SaveChangesAsync(). 11. Invalidate Output Cache tag "recipes". 12. Trả về HTTP 201 Created với RecipeDto.
Luồng thay thế / Ngoại lệ	A1 – Không có quyền Author/Admin: HTTP 401/403. A2 – CategoryId không tồn tại: HTTP 422 với lỗi "Category không hợp lệ". A3 – Slug đã tồn tại (title trùng): HTTP 409 Conflict.
HTTP Method & Endpoint	POST /api/v1/recipes
Kết quả mong đợi	Recipe mới được tạo với Status = Draft, Slug được sinh tự động. Cache "recipes" bị invalidate.
HTTP Status Code trả về	201 Created – Tạo thành công. 401/403 – Chưa đăng nhập / Không có quyền. 409 Conflict – Slug đã tồn tại. 422 Unprocessable Entity – Dữ liệu không hợp lệ.

FR-RCP-004: Cập nhật Công thức [Author-Owner/Admin]

Mã yêu cầu	FR-RCP-004
Tên yêu cầu	Cập nhật Thông tin Công thức Nấu ăn
Nhóm chức năng	Module Quản lý Công thức Nấu ăn (FR-RCP)
Tác nhân	Tác giả sở hữu (Author – Owner) / Quản trị viên (Admin)
Mức ưu tiên (MoSCoW)	M – Must Have
Mô tả	Cập nhật thông tin của một công thức. Resource-Based Authorization được áp dụng: chỉ Author sở hữu recipe (AuthorId == currentUserId) hoặc Admin được phép. Concurrency control qua RowVersion (ETag pattern): client phải gửi RowVersion hiện tại trong If-Match header; nếu mismatch → conflict.
Điều kiện tiên quyết	1. Author/Admin đang đăng nhập. 2. Recipe với ID tương ứng tồn tại. 3. Client cung cấp RowVersion hợp lệ trong If-Match header (hoặc trong request body).
Luồng chính (Happy Path)	1. Author gửi PUT /api/v1/recipes/{id} với body: { title, description, categoryId, prepTime, cookTime, servings, difficulty, instructions, nutrition? }. 2. Kiểm tra xác thực. 3. UpdateRecipeCommand dispatch. 4. Lấy recipe từ database theo ID. 5. IAuthorizationService.AuthorizeAsync(user, recipe, Operations.Update) – kiểm tra resource-based auth. 6. Kiểm tra RowVersion: DbContext sẽ ném DbUpdateConcurrencyException nếu RowVersion mismatch. 7. Update các field của recipe entity qua domain method recipe.Update(...).

	8. Cập nhật Nutrition nếu có. 9. SaveChangesAsync() – nếu RowVersion mismatch tại đây → ném ConcurrencyException → HTTP 409. 10. Invalidate cache: EvictByTagAsync("recipes"), EvictByTagAsync(\$"recipe:{slug}"). 11. Trả về HTTP 200 OK với RecipeDto đã cập nhật.
Luồng thay thế / Ngoại lệ	A1 – Không phải owner (Author khác): HTTP 403 Forbidden. A2 – Concurrency conflict (RowVersion mismatch): HTTP 409 Conflict – "Dữ liệu đã bị thay đổi bởi người dùng khác." A3 – ID không tồn tại: HTTP 404.
HTTP Method & Endpoint	PUT /api/v1/recipes/{id:guid}
Kết quả mong đợi	Recipe được cập nhật, cache bị invalidate, trả về RecipeDto mới nhất.
HTTP Status Code trả về	200 OK. 403 Forbidden – Không phải owner. 404 Not Found. 409 Conflict – Concurrency hoặc slug trùng. 422 Unprocessable Entity.

FR-RCP-005: Xuất bản / Hủy Xuất bản Công thức

Mã yêu cầu	FR-RCP-005
Tên yêu cầu	Xuất bản (Publish) / Hủy Xuất bản (Unpublish) Công thức
Nhóm chức năng	Module Quản lý Công thức Nấu ăn (FR-RCP)
Tác nhân	Tác giả sở hữu (Author – Owner) / Quản trị viên (Admin)
Mức ưu tiên (MoSCoW)	M – Must Have
Mô tả	Thay đổi trạng thái công thức: Draft → Published (xuất bản) hoặc Published → Draft (hủy xuất bản). Business rule: KHÔNG thể publish nếu recipe không có ít nhất 1 bước thực hiện (Steps.Count > 0). Khi publish, Recipe trở nên công khai và được đưa vào index tìm kiếm.
Điều kiện tiên quyết	1. Recipe tồn tại, người dùng là owner hoặc Admin. 2. Để publish: recipe phải có ít nhất 1 RecipeStep.
Luồng chính (Happy Path)	1. Author gửi PATCH /api/v1/recipes/{id}/publish (để xuất bản) hoặc PATCH /api/v1/recipes/{id}/unpublish. 2. PublishRecipeCommand dispatch với isPublish = true/false. 3. Kiểm tra resource-based authorization. 4. Gọi domain method: recipe.Publish() hoặc recipe.Unpublish(). 5. recipe.Publish() kiểm tra: Steps.Count == 0 → Throw DomainException("Recipe phải có ít nhất 1 bước thực hiện."). 6. Set Status = Published/Draft, UpdatedAt = DateTime.UtcNow. 7. SaveChangesAsync(), invalidate cache. 8. Trả về HTTP 200 OK với RecipeDto.
Luồng thay thế / Ngoại lệ	A1 – Recipe không có bước thực hiện: HTTP 422 với DomainException message. A2 – Recipe đã ở trạng thái mong muốn: Idempotent, trả về HTTP 200 OK.

HTTP Method & Endpoint	PATCH /api/v1/recipes/{id:guid}/publish PATCH /api/v1/recipes/{id:guid}/unpublish
Kết quả mong đợi	Status recipe được thay đổi thành Published hoặc Draft. Cache bị invalidate.
HTTP Status Code trả về	200 OK – Thành công. 403 Forbidden. 404 Not Found. 422 Unprocessable Entity – Thiếu steps.

FR-RCP-006: Lưu trữ Công thức (Archive)

Mã yêu cầu	FR-RCP-006
Tên yêu cầu	Lưu trữ Công thức (Archive / Unarchive)
Nhóm chức năng	Module Quản lý Công thức Nấu ăn (FR-RCP)
Tác nhân	Tác giả sở hữu / Quản trị viên (Admin)
Mức ưu tiên (MoSCoW)	S – Should Have
Mô tả	Chuyển Recipe sang trạng thái Archived. Recipe Archived không hiển thị trong danh sách công khai nhưng không bị xóa khỏi database (soft hide). Hữu ích để ẩn recipe cũ không còn phù hợp mà không mất dữ liệu.
Điều kiện tiên quyết	1. Recipe tồn tại, người dùng có quyền.
Luồng chính (Happy Path)	1. Author gửi PATCH /api/v1/recipes/{id}/archive. 2. Kiểm tra authorization. 3. recipe.Archive() → Status = Archived. 4. SaveChangesAsync(), invalidate cache. 5. HTTP 200 OK.
Luồng thay thế / Ngoại lệ	A1 – ID không tồn tại: HTTP 404. A2 – Không có quyền: HTTP 403.
HTTP Method & Endpoint	PATCH /api/v1/recipes/{id:guid}/archive
Kết quả mong đợi	Status = Archived. Recipe không còn xuất hiện trong public listing.
HTTP Status Code trả về	200 OK. 403 Forbidden. 404 Not Found.

FR-RCP-007: Xóa Công thức [Author-Owner/Admin]

Mã yêu cầu	FR-RCP-007
Tên yêu cầu	Xóa Vĩnh viễn Công thức Nấu ăn
Nhóm chức năng	Module Quản lý Công thức Nấu ăn (FR-RCP)
Tác nhân	Tác giả sở hữu (Author – Owner) / Quản trị viên (Admin)
Mức ưu tiên (MoSCoW)	M – Must Have
Mô tả	Xóa vĩnh viễn một công thức và tất cả dữ liệu liên quan (cascade delete: Steps, Ingredients, Images). Các file ảnh trên MinIO được xóa

	bất đồng bộ qua Hangfire fire-and-forget job để tránh blocking HTTP response. Đây là hard delete (không dùng soft delete pattern cho recipe).
Điều kiện tiên quyết	1. Recipe tồn tại. 2. Người dùng là owner hoặc Admin.
Luồng chính (Happy Path)	1. Author/Admin gửi DELETE /api/v1/recipes/{id}. 2. Kiểm tra xác thực và resource-based authorization. 3. Lấy danh sách URL ảnh từ recipe.Images. 4. _unitOfWork.Recipes.Remove(recipe), SaveChangesAsync() – cascade delete Steps, Ingredients, Images trong database. 5. Với mỗi imageUrl: BackgroundJob.Enqueue<IFileStorageService>(svc => svc.DeleteAsync(url)) – xóa ảnh trên MinIO bất đồng bộ. 6. EvictByTagAsync("recipes"), EvictByTagAsync(\$"recipe:{recipe.Slug}") – invalidate cache. 7. Trả về HTTP 204 No Content.
Luồng thay thế / Ngoại lệ	A1 – ID không tồn tại: HTTP 404. A2 – Không phải owner: HTTP 403. A3 – Xóa MinIO file thất bại (job retry): Hangfire tự động retry 3 lần. Nếu vẫn fail, log error nhưng không ảnh hưởng response đã trả về.
HTTP Method & Endpoint	DELETE /api/v1/recipes/{id:guid}
Kết quả mong đợi	Recipe và tất cả child entities bị xóa khỏi database. Ảnh trên MinIO được lên lịch xóa qua Hangfire.
HTTP Status Code trả về	204 No Content – Xóa thành công. 403 Forbidden. 404 Not Found.

FR-RCP-008: Quản lý Ảnh Công thức (Upload / Set Primary / Delete)

Mã yêu cầu	FR-RCP-008
Tên yêu cầu	Upload Ảnh, Đặt Ảnh Chính, Xóa Ảnh Công thức
Nhóm chức năng	Module Quản lý Công thức Nấu ăn (FR-RCP)
Tác nhân	Tác giả sở hữu / Quản trị viên (Admin)
Mức ưu tiên (MoSCoW)	M – Must Have
Mô tả	Author quản lý ảnh minh họa cho công thức của mình. Upload dùng multipart/form-data. Ảnh được lưu trên MinIO với path: recipes/{recipeId}/{uuid}.{ext}. Ảnh đầu tiên tự động được đặt làm ảnh chính (IsPrimary = true). Hỗ trợ 3 thao tác: Upload (POST), đặt ảnh chính (PATCH primary), Xóa (DELETE). Validation bắt buộc: MIME type (image/jpeg, image/png, image/webp, image/avif) và kích thước tối đa 5MB.
Điều kiện tiên quyết	1. Author/Admin đang đăng nhập. 2. Recipe tồn tại và người dùng có quyền.
Luồng chính (Happy Path)	--- UPLOAD --- 1. POST /api/v1/recipes/{id}/images với multipart/form-data chứa field "file".

	2. Validate MIME type: chỉ chấp nhận image/jpeg, image/png, image/webp, image/avif. 3. Validate kích thước: file.Length <= 5*1024*1024 bytes (5MB). 4. Validate magic bytes: đọc 4 bytes đầu để xác nhận định dạng thực sự (JPEG: FF D8 FF; PNG: 89 50 4E 47). 5. IFileStorageService.UploadAsync(file, "recipes/{id}") → trả về URL công khai. 6. RecipeImage.Create(url, altText, isPrimary: !recipe.Images.Any()) → thêm vào recipe. 7. SaveChangesAsync(), invalidate cache. 8. HTTP 201 Created với { url, isPrimary }. --- SET PRIMARY IMAGE --- 9. PATCH /api/v1/recipes/{id}/images/{imgId}/primary. 10. Tìm image theo imgId, đặt IsPrimary = true, đặt tất cả ảnh khác IsPrimary = false. 11. HTTP 200 OK. --- DELETE IMAGE --- 12. DELETE /api/v1/recipes/{id}/images/{imgId}. 13. Xóa entity khỏi database. 14. BackgroundJob.Enqueue xóa file trên MinIO. 15. Nếu ảnh bị xóa là IsPrimary và còn ảnh khác: tự động đặt ảnh đầu tiên còn lại làm primary. 16. HTTP 204 No Content.
Luồng thay thế / Ngoại lệ	A1 – MIME type không hợp lệ: HTTP 400 Bad Request. A2 – File vượt quá 5MB: HTTP 400 với message "Kích thước file vượt quá giới hạn 5MB." A3 – Magic bytes không khớp MIME type: HTTP 400 "File không hợp lệ." A4 – MinIO không khả dụng: HTTP 503 Service Unavailable.
HTTP Method & Endpoint	POST /api/v1/recipes/{id}/images PATCH /api/v1/recipes/{id}/images/{imgId}/primary DELETE /api/v1/recipes/{id}/images/{imgId}
Kết quả mong đợi	Ảnh được upload lên MinIO, URL lưu vào database. IsPrimary được quản lý chính xác.
HTTP Status Code trả về	Upload: 201 Created. Set Primary: 200 OK. Delete: 204 No Content. 400 Bad Request – File không hợp lệ. 403/404 – Lỗi quyền/không tìm thấy.

FR-RCP-009: Quản lý Nguyên liệu (CRUD RecipeIngredient)

Mã yêu cầu	FR-RCP-009
Tên yêu cầu	Thêm / Cập nhật / Xóa Nguyên liệu Công thức
Nhóm chức năng	Module Quản lý Công thức Nấu ăn (FR-RCP)
Tác nhân	Tác giả sở hữu / Quản trị viên (Admin)

Mức ưu tiên (MoSCoW)	M – Must Have
Mô tả	Author quản lý danh sách nguyên liệu (RecipeIngredient) của công thức. Mỗi nguyên liệu có: Name (tên), Quantity (số lượng), Unit (đơn vị: gram/ml/muỗng/cái/củ...), Notes (ghi chú tùy chọn), SortOrder (thứ tự hiển thị). Endpoint hỗ trợ thêm mới (POST), cập nhật (PUT), xóa (DELETE) từng nguyên liệu riêng lẻ.
Điều kiện tiên quyết	1. Recipe tồn tại và người dùng có quyền. 2. Quantity > 0, Unit không rỗng, Name 1–100 ký tự.
Luồng chính (Happy Path)	--- THÊM NGUYÊN LIỆU --- 1. POST /api/v1/recipes/{id}/ingredients với body: { name, quantity, unit, notes?, sortOrder? }. 2. Validate, tạo RecipeIngredient.Create(recipeId, name, qty, unit, notes, sortOrder). 3. _unitOfWork.Recipes (qua navigation) thêm ingredient, SaveChangesAsync(). 4. HTTP 201 Created. --- CẬP NHẬT NGUYÊN LIỆU --- 5. PUT /api/v1/recipes/{id}/ingredients/{ingId} với body fields cần cập nhật. 6. Tìm ingredient, cập nhật, SaveChangesAsync(). HTTP 200 OK. --- XÓA NGUYÊN LIỆU --- 7. DELETE /api/v1/recipes/{id}/ingredients/{ingId}. 8. Xóa entity, SaveChangesAsync(). HTTP 204 No Content.
Luồng thay thế / Ngoại lệ	A1 – Recipe/Ingredient không tồn tại: HTTP 404. A2 – Không có quyền: HTTP 403. A3 – Dữ liệu không hợp lệ: HTTP 422.
HTTP Method & Endpoint	POST/PUT/DELETE /api/v1/recipes/{id}/ingredients/{ingId?}
Kết quả mong đợi	Danh sách nguyên liệu được cập nhật chính xác. Cache bị invalidate.
HTTP Status Code trả về	201/200/204 – Thành công. 403/404/422 – Lỗi tương ứng.

FR-RCP-010: Quản lý Các bước Thực hiện (CRUD RecipeStep)

Mã yêu cầu	FR-RCP-010
Tên yêu cầu	Thêm / Cập nhật / Xóa Bước Thực hiện Công thức
Nhóm chức năng	Module Quản lý Công thức Nấu ăn (FR-RCP)
Tác nhân	Tác giả sở hữu / Quản trị viên (Admin)
Mức ưu tiên (MoSCoW)	M – Must Have
Mô tả	Author quản lý các bước thực hiện (RecipeStep) của công thức. Mỗi bước có: StepNumber (thứ tự, tự động tăng), Description (mô tả bước), DurationMinutes (thời gian ước tính cho bước, tùy chọn), imageUrl (ảnh minh họa cho bước riêng, tùy chọn). Khi xóa một bước, hệ thống

	tự động renumber các bước còn lại để đảm bảo StepNumber liên tục (1, 2, 3...).
Điều kiện tiên quyết	1. Recipe tồn tại, người dùng có quyền. 2. Description không rỗng, tối đa 2000 ký tự.
Luồng chính (Happy Path)	1. POST /api/v1/recipes/{id}/steps với body: { description, durationMinutes?, imageUrl? }. 2. StepNumber = recipe.Steps.Max(s => s.StepNumber) + 1 (hoặc 1 nếu chưa có bước nào). 3. RecipeStep.Create(recipeId, stepNumber, description, durationMinutes). 4. SaveChangesAsync(). HTTP 201 Created. --- XÓA BƯỚC --- 5. DELETE /api/v1/recipes/{id}/steps/{stepId}. 6. Xóa step, sau đó renumber: cập nhật StepNumber của tất cả steps còn lại theo thứ tự. 7. SaveChangesAsync(). HTTP 204 No Content.
Luồng thay thế / Ngoại lệ	A1 – Recipe không tồn tại: HTTP 404. A2 – Không có quyền: HTTP 403.
HTTP Method & Endpoint	POST/PUT/DELETE /api/v1/recipes/{id}/steps/{stepId?}
Kết quả mong đợi	Danh sách steps được cập nhật với StepNumber liên tục. Cache bị invalidate.
HTTP Status Code trả về	201/200/204 – Thành công. 403/404/422 – Lỗi.

3.4. Module Tìm kiếm và Phân trang (FR-SRCH)

FR-SRCH-001: Tìm kiếm Toàn văn bản (Full-Text Search)

Mã yêu cầu	FR-SRCH-001
Tên yêu cầu	Tìm kiếm Toàn văn bản Công thức (Full-Text Search)
Nhóm chức năng	Module Tìm kiếm và Phân trang (FR-SRCH)
Tác nhân	Tất cả (Guest / Author / Admin)
Mức ưu tiên (MoSCoW)	M – Must Have
Mô tả	Hệ thống cung cấp tính năng tìm kiếm toàn văn bản (FTS) cho công thức sử dụng PostgreSQL tsvector/tsquery với cấu hình tiếng Việt. Trường SearchVector (computed column) được tự động cập nhật bởi PostgreSQL trigger khi Title hoặc Description thay đổi. Kết quả được xếp hạng bởi ts_rank(). Hỗ trợ tìm kiếm gần đúng với unaccent extension (bỏ dấu tiếng Việt: "pho" tìm được "phở").
Điều kiện tiên quyết	1. PostgreSQL extensions unaccent và pg_trgm đã được install. 2. GIN index trên cột SearchVector đã được tạo. 3. Tham số q không rỗng, tối thiểu 2 ký tự.

Luồng chính (Happy Path)	- Client gửi GET <code>/api/v1/recipes/search?q=pho+bo&page=1&pageSize=10</code>. <code>SearchRecipesQuery</code> dispatch với <code>SearchTerm = "pho bo"</code>, <code>Page = 1</code>, <code>PageSize = 10</code>. - Handler xây dựng <code>tsquery</code> từ search terms: <code>"pho:* & bo:*" (prefix matching)</code>. - LINQ query với EF Core: <code>.Where(r => r.SearchVector.Matches(EF.Functions.ToTsQuery("vietnamese", query)))</code>. - Apply ORDER BY <code>ts_rank(SearchVector, query) DESC</code> để kết quả liên quan nhất lên đầu. - Chỉ trả về <code>Status == Published</code> recipes. - Apply pagination, trả về <code>PagedResult<RecipeSummaryDto></code> với field <code>relevanceScore</code>. - Kết quả KHÔNG cache (vì query string đa dạng) hoặc cache ngắn (5 phút) với vary by query.
Luồng thay thế / Ngoại lệ	A1 – Query rỗng hoặc < 2 ký tự: HTTP 422. A2 – Không tìm thấy kết quả: HTTP 200 với <code>items = []</code> và message gợi ý. A3 – Ký tự đặc biệt trong query (SQL injection attempt): EF Core parameterize tự động; <code>tsquery</code> sanitization loại bỏ ký tự nguy hiểm.
HTTP Method & Endpoint	GET <code>/api/v1/recipes/search?q={searchTerm}&page={n}&pageSize={n}</code>
Kết quả mong đợi	<code>PagedResult<RecipeSummaryDto></code> được xếp hạng theo độ liên quan (<code>ts_rank</code>). Hỗ trợ tìm kiếm không dấu tiếng Việt.
HTTP Status Code trả về	200 OK – Thành công (kể cả kết quả rỗng). 422 – Query không hợp lệ.

FR-SRCH-002/003/004: Lọc, Sắp xếp và Phân trang (Tóm tắt)

Ba FR còn lại của module Search được tích hợp sẵn vào FR-RCP-001 và FR-SRCH-001. Bảng tóm tắt:

Mã FR	Tên	Tham số Query	Mô tả
FR-SRCH-002	Lọc công thức	<code>categoryId={guid}</code> <code>difficulty={Easy Medium Hard}</code> <code>maxCookTime={minutes}</code> <code>minServings={n}</code>	Lọc kết quả theo một hoặc nhiều tiêu chí. Các filter kết hợp bằng AND logic.
FR-SRCH-003	Sắp xếp kết quả	<code>sort={field}</code> VD: <code>sort=createdAt</code> (ASC) <code>sort=-createdAt</code> (DESC) <code>sort=title</code> , <code>sort=-cookTime</code>	Tiền tố "-" = descending. Mặc định: <code>sort=-createdAt</code> (mới nhất trước).
FR-SRCH-004	Phân trang (Offset-based)	<code>page={n}</code> (default: 1) <code>pageSize={n}</code> (default: 12, max: 50)	Offset-based pagination (SKIP/TAKE). Response bao gồm <code>totalCount</code> , <code>totalPages</code> , <code>hasNextPage</code> , <code>hasPreviousPage</code> .

3.5. Module Quản lý Tập tin (FR-FILE)

Module xử lý tất cả thao tác với file binary trên hệ thống lưu trữ đối tượng (Object Storage) MinIO S3-compatible. Abstraction layer IFileStorageService cho phép swap implementation (MinIO ↔ AWS S3 ↔ local filesystem) mà không cần thay đổi Application Layer.

Mã FR	Tên	Mô tả	Ràng buộc kỹ thuật
FR-FILE-001	Upload File lên MinIO	IFileStorageService.UploadAsync(IFormFile, folder, ct) → string (public URL). Tạo unique filename = {folder}/{Guid.NewGuid()}{ext} để ngăn path traversal. Preserve MIME type gốc.	Max size: 5MB. MIME: JPEG/PNG/WebP/AVIF. Magic bytes validation. Bucket: "culinary-blog". Policy: public-read.
FR-FILE-002	Xóa File khỏi MinIO	IFileStorageService.DeleteAsync(fileUrl, ct). Trích xuất object name từ URL, gọi RemoveObjectAsync(). Thường được gọi từ Hangfire background job (fire-and-forget) sau khi xóa recipe.	Nếu object không tồn tại trên MinIO → không throw exception (idempotent). Lỗi kết nối MinIO → Hangfire retry tối đa 3 lần.

3.6. Module Background Jobs (FR-JOB)

Module xử lý các tác vụ nền không đồng bộ sử dụng Hangfire. Hangfire chạy in-process trong .NET API và sử dụng PostgreSQL làm persistent storage cho job queue. Dashboard quản lý jobs tại /hangfire (chỉ Admin). Hỗ trợ 3 loại job: Fire-and-forget (chạy ngay), Delayed (chạy sau N giây/phút) và Recurring (lịch cron).

Mã FR	Tên Job	Loại	Trigger	Mô tả	Retry Policy
FR-JOB-001	Welcome Email Job	Fire-and-forget	Sau FR-AUTH-001 thành công (BackgroundJob.Enqueue)	Gửi email HTML chào mừng đến địa chỉ email vừa đăng ký. Email template bao gồm: tên người dùng, link kích hoạt email (nếu cần), link đến ứng dụng.	Tự động retry 3 lần với exponential backoff (1 phút, 5 phút, 30 phút). Sau 3 lần fail → chuyển sang Failed state, log error.
FR-JOB-002	Image Resize / Thumbnail Job	Fire-and-forget	Sau FR-RCP-008 upload ảnh thành công	Tạo thumbnail (300x300px) và medium image (800x600px) từ ảnh gốc. Lưu cả 3 phiên bản lên MinIO. Cập nhật URLs vào database.	Retry 3 lần. Nếu fail: ảnh gốc vẫn hiển thị, chỉ thiếu thumbnail.
FR-JOB-003	Sitemap Generation Job	Recurring	Hàng ngày lúc 02:00 AM UTC (cron: "0 2 * * *")	Tạo file sitemap.xml chứa URL tất cả Published recipes, categories và pages tĩnh. Upload sitemap.xml lên MinIO hoặc lưu	Retry 2 lần nếu fail. Log kết quả (số URL trong sitemap) qua Serilog.

Mã FR	Tên Job	Loại	Trigger	Mô tả	Retry Policy
				vào wwwroot. Gửi thông báo đến Google Search Console (ping).	

3.7. Module Quan sát Hệ thống (FR-OBS)

Module cung cấp khả năng quan sát (Observability) toàn diện theo ba trụ cột: Logging (Serilog), Metrics (OpenTelemetry), và Distributed Tracing (OpenTelemetry). Đây là yêu cầu bắt buộc cho production deployment.

Mã FR	Tên	Mô tả	Kỹ thuật / Công cụ
FR-OBS-001	Health Check Endpoints	Hệ thống cung cấp 3 endpoint health check với mục đích khác nhau: • GET /health – tổng hợp tất cả components (database, Redis, MinIO). • GET /health/live – Liveness probe (chỉ kiểm tra process còn sống). • GET /health/ready – Readiness probe (kiểm tra kết nối database và Redis).	IHealthCheck,.AspNetCore.HealthChecks.Npgsql,.AspNetCore.HealthChecks.Redis,.AspNetCore.HealthChecks.Minio. Liveness chỉ trả healthy. Readiness fail khi DB/Redis down → Kubernetes/Nginx ngừng route traffic.
FR-OBS-002	Structured Logging	Mọi HTTP request được log với: CorrelationId (X-Correlation-ID header), HTTP method/path/status, elapsed time (ms), UserId (khi đã xác thực). MediatR Pipeline Behavior (LoggingBehavior) log tất cả Commands/Queries vào. Performance alert khi request > 500ms.	Serilog + CorrelationIdMiddleware. Sinks: Console (structured JSON), File (rolling daily), Seq (development). Log levels: Debug (development), Information (production), Warning/Error (luôn luôn).
FR-OBS-003	Distributed Tracing & Metrics	OpenTelemetry instrumentation cho: HTTP request traces (ActivitySource), EF Core database operation traces, custom business metrics (recipe created/published count). Traces được export đến Seq (development) hoặc Jaeger/Grafana Tempo (production).	OpenTelemetry .NET SDK, OTLP exporter. Activity.TraceId được include trong structured log (log correlation với trace). Metrics: request count, duration histogram, error rate.

4. Yêu cầu Phi Chức năng (NFR)

Phần này mô tả các thuộc tính chất lượng hệ thống theo mô hình ISO/IEC 25010 (FURPS+). Mỗi yêu cầu phi chức năng được gán mã định danh, mức ưu tiên và tiêu chí đo lường định lượng cụ thể. Các NFR này ràng buộc thiết kế kiến trúc và lựa chọn công nghệ toàn bộ hệ thống.

Mã NFR	Danh mục	Số yêu cầu	Ưu tiên
NFR-PERF	Hiệu năng (Performance)	5	Cao
NFR-SEC	Bảo mật (Security)	6	Rất cao
NFR-USE	Khả năng sử dụng (Usability)	4	Trung bình
NFR-REL	Độ tin cậy (Reliability)	3	Cao
NFR-MAINT	Khả năng bảo trì (Maintainability)	4	Trung bình
NFR-SCALE	Khả năng mở rộng (Scalability)	3	Cao
NFR-SEO	Tối ưu SEO (SEO)	4	Cao

4.1. Hiệu năng (NFR-PERF)

Toàn bộ các chỉ số hiệu năng được đo trong môi trường production với tải thực tế. Các ngưỡng dưới đây áp dụng cho trường hợp cache warm (Redis hit rate $\geq 80\%$).

NFR-PERF-001 Response Time API	Thời gian phản hồi API: • $p_{50} \leq 150\text{ms}$ — cho tất cả GET endpoints với dữ liệu cache. • $p_{95} \leq 500\text{ms}$ — cho tất cả API endpoints (kể cả write operations). • $p_{99} \leq 1000\text{ms}$ — không vượt quá 1 giây trong mọi trường hợp. Đo bằng: OpenTelemetry + Grafana / k6 load test.
NFR-PERF-002 Throughput	Hệ thống xử lý đồng thời ≥ 100 concurrent users mà không degradation: • Trên phần cứng: 2 vCPU, 4GB RAM (single instance). • Horizontal scaling: thêm instance tăng tuyến tính. Đo bằng: k6 smoke test $\rightarrow$ load test $\rightarrow$ stress test.
NFR-PERF-003 Cache Effectiveness	Redis Cache hit rate $\geq 80\%$ trong điều kiện steady-state. Các đối tượng cache: • Category list: TTL = 30 phút (ít thay đổi). • Recipe detail: TTL = 5 phút (cache-aside pattern). • Search results: TTL = 1 phút. Cache invalidation: Event-driven — xóa cache khi Create/Update/Delete.
NFR-PERF-004 Database Query	Tất cả queries đến PostgreSQL: • Không có N+1 query problem — bắt buộc dùng .Include()/ThenInclude() và projection. • Index: đảm bảo mọi WHERE/ORDER BY column đều có B-tree index tương ứng. • Slow query log: cảnh báo khi

	query > 100ms (Serilog performance behavior). • EXPLAIN ANALYZE: phải pass review trước khi merge.
NFR-PERF-005 Frontend Performance (Core Web Vitals)	Next.js frontend đạt chuẩn Google Core Web Vitals (đo bằng Lighthouse CI): • LCP (Largest Contentful Paint) ≤ 2.5s. • CLS (Cumulative Layout Shift) ≤ 0.1. • INP (Interaction to Next Paint) ≤ 200ms. • First Load JS Bundle ≤ 200KB (gzipped). Kỹ thuật: ISR (Incremental Static Regeneration), Image Optimization (next/image), Code Splitting.

4.2. Bảo mật (NFR-SEC)

Toàn bộ yêu cầu bảo mật tuân thủ OWASP Top 10 (2021) và được kiểm thử qua security review trước khi release production.

NFR-SEC-001 Password & Hashing	Mật khẩu phải được hash bằng ASP.NET Core Identity mặc định (PBKDF2-HMACSHA512, iteration count ≥ 100.000). Không bao giờ lưu plaintext password. Yêu cầu độ phức tạp: ≥ 8 ký tự, chứa ít nhất 1 chữ hoa + 1 chữ thường + 1 số + 1 ký tự đặc biệt (cấu hình qua IdentityOptions.Password).
NFR-SEC-002 JWT Token Security	Access Token: JWT signed bằng HS256, TTL = 15 phút, claim: userId, email, roles, jti. Refresh Token: 128-bit cryptographically secure random bytes, hash SHA-256 trước khi lưu DB, TTL = 7 ngày. Rotation: Refresh token bị revoke ngay sau khi dùng, cấp token mới (Refresh Token Rotation). Detection: Nếu refresh token đã bị revoke được dùng lại → revoke toàn bộ family (Reuse Detection).
NFR-SEC-003 Rate Limiting	Giới hạn yêu cầu theo IP để ngăn brute force và DDoS: • Auth endpoints (/auth/*): 10 request/phút/IP. • API chung: 100 request/phút/IP. • Upload endpoints: 5 request/phút/IP. Implementation: ASP.NET Core Rate Limiting middleware (Fixed Window, sliding window cho auth). HTTP 429 khi vượt giới hạn với Retry-After header.
NFR-SEC-004 Input Validation & File Upload Security	Toàn bộ input được validate tại Application Layer (FluentValidation) TRƯỚC khi xử lý: • SQL Injection: EF Core parameterized queries (không raw SQL với user input). • XSS: Input sanitization + Content-Security-Policy header. • MIME Validation: Đọc magic bytes (không tin vào Content-Type header) khi upload. • File size: Kiểm tra trước khi read stream (không buffer toàn bộ vào memory trước). • Path Traversal: GUID-based filename generation (không dùng tên file của user).
NFR-SEC-005 HTTPS & CORS	Toàn bộ traffic phải qua HTTPS (TLS 1.2+): • Nginx: redirect HTTP → HTTPS, HSTS header (max-age=31536000). • CORS Policy: Chỉ cho phép origin được cấu hình qua appsettings (không wildcard *). • Allowed Origins: http://localhost:3000 (dev), https://domain.com (prod). • Cookie: SameSite=Strict, Secure=true (nếu dùng cookie cho refresh token).

NFR-SEC-006 Authorization & Resource Ownership	Kiểm tra phân quyền tại Application Layer (không chỉ ở Presentation Layer): • Authorization Handler: RecipeAuthorizationHandler xác minh ResourceOwnership (Author chỉ xóa recipe của mình). • Role-based policies: "AuthorPolicy", "AdminPolicy" (không hardcode role string). • Sensitive endpoints (DELETE, PATCH publish): double-check user ID trước khi commit. • Audit trail: Log mọi write operation với userId + timestamp (Serilog).
NFR-SEC-007 Secrets Management	Không bao giờ commit secrets vào Git: • Development: ASP.NET Core User Secrets (dotnet user-secrets). • Production: Environment variables (Docker Compose env_file / Kubernetes Secrets). • Rotation: Khuyến nghị rotate JWT signing key mỗi 90 ngày. • Scanning: Pre-commit hook kiểm tra với truffleHog/gitLeaks.

4.3. Khả năng Sử dụng (NFR-USE)

NFR-USE-001 Responsive Design	Giao diện hiển thị chính xác trên tất cả breakpoints: • Mobile: 320px – 767px (single column, touch-friendly). • Tablet: 768px – 1199px (2-column grid). • Desktop: ≥ 1200px (full layout). Framework: Tailwind CSS utility-first. Không sử dụng CSS framework override. Kiểm thử: Chrome DevTools responsive mode + BrowserStack (iOS, Android).
NFR-USE-002 Accessibility (a11y)	Tuân thủ WCAG 2.1 Level AA: • Semantic HTML5: <article>, <nav>, <main>, <aside>. • ARIA attributes: aria-label, aria-expanded, role trên interactive elements. • Keyboard navigation: tất cả chức năng dùng được bằng bàn phím (Tab, Enter, Escape). • Color contrast ratio ≥ 4.5:1 (text) và ≥ 3:1 (UI components). • Screen reader: test với NVDA (Windows) và VoiceOver (macOS/iOS).
NFR-USE-003 Error Messages	Thông báo lỗi phải rõ ràng và actionable: • API: trả về RFC 7807 Problem Details (type, title, status, detail, errors{}). • Frontend: hiển thị ngay bên cạnh field lỗi (React Hook Form inline validation). • Server errors (5xx): hiển thị thông báo thân thiện, không lộ stack trace. • I18n-ready: error messages sử dụng error code (không hardcode tiếng Việt/Anh).
NFR-USE-004 Loading States	Mọi async operation phải có visual feedback: • Loading skeleton: hiển thị trong khi fetch data (không blank screen). • Optimistic update: UI cập nhật ngay, rollback nếu API fail. • Toast notification: xác nhận thành công/thất bại sau write operation. • Progress indicator: upload ảnh hiển thị progress bar (%) realtime.

4.4. Độ tin cậy (NFR-REL)

NFR-REL-001 Uptime SLA	Hệ thống có uptime $\geq 99.5\%$ (≈ 3.65 giờ downtime/năm). • Maintenance window: công bố trước 48 giờ qua banner thông báo. • Health check: /health/ready probe mỗi 10 giây (Kubernetes readiness probe). • Monitoring: Uptime Robot / Better Uptime gửi alert khi down > 1 phút.
NFR-REL-002 Error Handling & Resilience	Hệ thống xử lý lỗi gracefully, không crash toàn bộ: • Global Exception Handler Middleware: bắt tất cả unhandled exceptions $\rightarrow$ trả 500 Problem Details + log. • Database connection pool: tự reconnect, timeout 30s. • Redis failover: nếu Redis down $\rightarrow$ fallback database (không cache), không throw exception. • Hangfire retry: mỗi job tối đa 3 retry với exponential backoff. • Circuit Breaker: (tùy chọn nâng cao) Polly cho external HTTP calls.
NFR-REL-003 Data Durability	Dữ liệu không bị mất trong trường hợp restart hoặc crash: • PostgreSQL WAL (Write-Ahead Logging): đảm bảo ACID. • Backup: pg_dump tự động hàng ngày lúc 03:00 AM, lưu 30 ngày. • MinIO: dữ liệu file trên volume persistent (không ephemeral container storage). • Refresh tokens: lưu DB (không Redis) để survive restart. • Soft delete: Recipe được đánh dấu IsDeleted thay vì xóa vật lý (có thể khôi phục).

4.5. Khả năng Bảo trì (NFR-MAINT)

NFR-MAINT-001 Code Quality	Toàn bộ code phải pass static analysis trước khi merge: • .NET: SonarAnalyzer, StyleCop, EditorConfig (indent, naming conventions). • TypeScript/React: ESLint (Airbnb ruleset), Prettier. • Không có compiler warnings trong build CI. • Code review: ít nhất 1 reviewer phê duyệt Pull Request.
NFR-MAINT-002 Test Coverage	Độ phủ test tối thiểu: • Unit tests: $\geq 80\%$ line coverage (Application layer commands, queries, validators). • Integration tests: tất cả API endpoints có ít nhất 1 happy path + 1 error case. • E2E tests: 5 critical user flows (register, login, create recipe, publish, search). Tool: xUnit (backend), Jest + Testing Library (frontend), Playwright (E2E).
NFR-MAINT-003 Documentation	Tài liệu kỹ thuật bắt buộc: • README.md: hướng dẫn setup dev environment (Docker Compose) trong < 5 phút. • API documentation: tự động sinh từ XML comments + Scalar/Swagger UI tại /scalar. • Architecture Decision Records (ADR): ghi lại mọi quyết định kiến trúc quan trọng. • CHANGELOG.md: cập nhật mỗi release (theo Keep a Changelog + SemVer).
NFR-MAINT-004 Clean Architecture Compliance	Tuân thủ nghiêm ngặt dependency rules của Clean Architecture: • Domain layer: KHÔNG dependency vào bất kỳ layer nào khác. Không có nuget packages ngoài FluentValidation. • Application layer: chỉ depend vào Domain. KHÔNG reference Infrastructure. • Infrastructure layer: depend vào Application (implements interfaces). • Vi phạm: được phát hiện qua ArchUnit.NET tests hoặc custom Architecture test

project. • CQRS: Commands thay đổi state, Queries đọc data — không trộn lẫn.

4.6. Khả năng Mở rộng (NFR-SCALE)

NFR-SCALE-001 Stateless Backend	API được thiết kế stateless để hỗ trợ horizontal scaling: • JWT authentication (không session server-side). • Distributed cache (Redis, không in-memory IMemoryCache) cho mọi shared state. • Distributed lock (RedLock) cho các tác vụ singleton (sitemap generation). • Hangfire: chạy với multiple workers (IBackgroundJobServer), PostgreSQL làm shared queue.
NFR-SCALE-002 Database Scaling	Chiến lược database scaling: • Connection pooling: Npgsql built-in pool (max 100 connections/instance). • Read replica (tùy chọn): EF Core split queries + IQueryable routing qua IDbContextFactory. • Index strategy: B-tree cho equality/range, GIN cho full-text search (tsvector). • Table partitioning: (nâng cao) partition Recipe by CreatedAt khi > 1 triệu rows.
NFR-SCALE-003 Infrastructure Scaling	Hạ tầng có thể scale theo chiều ngang: • Docker: mỗi service là container riêng biệt (API, Postgres, Redis, MinIO, Nginx). • Nginx: load balancer upstream pool cho nhiều API instances. • MinIO: Distributed Mode (4+ nodes) cho production storage scaling. • CDN: static assets (Next.js _next/static) được serve qua CDN (Cloudflare).

4.7. Tối ưu SEO (NFR-SEO)

NFR-SEO-001 Structured Data	Mỗi trang công thức nấu ăn phải có JSON-LD Schema.org Recipe markup: • @type: "Recipe" • Thuộc tính: name, description, image, author, datePublished, prepTime, cookTime, totalTime, recipeYield, recipeIngredient[], recipeInstructions[], nutrition. • Validate: Google Rich Results Test — phải pass 100%. • Kết quả: Rich Snippets trên Google Search (star rating, time, ingredients).
NFR-SEO-002 Meta Tags & Open Graph	Mỗi trang phải có đầy đủ: • <title>: "{Recipe Name} Culinary Blog" (≤ 60 ký tự). • <meta name="description">: mô tả 150–160 ký tự. • Open Graph: og:title, og:description, og:image (1200×630px), og:url, og:type. • Twitter Card: summary_large_image. • Canonical URL: tránh duplicate content (slug-based URL). • Robots: index, follow (published) noindex (draft/archived).
NFR-SEO-003 Sitemap & Robots	Sitemap XML tự động: • Sinh bởi FR-JOB-003 (Hangfire Recurring Job, hàng ngày 02:00 AM UTC). • Bao gồm: tất cả Published recipes + category pages + trang tĩnh. • Format: sitemap.xml chuẩn, có <loc>, <lastmod>, <changefreq>, <priority>. • Robots.txt: cho phép tất cả crawlers, khai báo

	Sitemap URL. • Ping Google Search Console sau khi update sitemap.
NFR-SEO-004 URL Structure	URL phải thân thiện SEO: • Recipes: /recipes/{slug} — slug là chữ thường, gạch nối, không dấu. • Categories: /categories/{slug}. • Slug generation: tự động từ title, unique, không thay đổi sau khi publish. • Redirect: Nếu slug thay đổi (draft) → 301 redirect từ slug cũ sang slug mới. • Không dùng query params cho nội dung chính (chỉ dùng cho filter/sort/pagination).

5. Yêu cầu Giao diện Ngoài

Chương này mô tả tất cả giao diện giữa hệ thống Culinary Blog với các thực thể bên ngoài: người dùng cuối, phần cứng, phần mềm bên thứ ba và giao tiếp mạng. Mọi giao tiếp đều qua HTTPS (TLS 1.2+) trong môi trường production.

5.1. Giao diện Người dùng (UI)

Hệ thống cung cấp giao diện web duy nhất trên nền Next.js App Router, hoạt động như Single Page Application (SPA) với Server-Side Rendering (SSR) và Incremental Static Regeneration (ISR).

Màn hình / Route	Mô tả	Loại Rendering	Yêu cầu Auth
/	Trang chủ: danh sách recipe nổi bật + categories	ISR (revalidate=3600)	Không
/recipes	Danh sách tất cả recipes với filter/sort/search	SSR (dynamic)	Không
/recipes/[slug]	Chi tiết recipe: ingredients, steps, nutrition, JSON-LD	ISR (revalidate=300)	Không
/categories	Danh sách category	ISR (revalidate=3600)	Không
/categories/[slug]	Danh sách recipe theo category	ISR (revalidate=600)	Không
/auth/login	Form đăng nhập (email/password + Google OAuth button)	CSR	Không (redirect nếu đã login)
/auth/register	Form đăng ký tài khoản mới	CSR	Không
/dashboard	Trang tổng quan của Author/Admin	CSR	Bắt buộc (Author/Admin)
/dashboard/recipes	Quản lý danh sách recipe của user	CSR	Bắt buộc
/dashboard/recipes/new	Form tạo recipe mới (multi-step wizard)	CSR	Bắt buộc (Author/Admin)
/dashboard/recipes/[id]/edit	Form chỉnh sửa recipe	CSR	Bắt buộc (Owner/Admin)
/dashboard/categories	Quản lý categories (chỉ Admin)	CSR	Bắt buộc (Admin)

Màn hình / Route	Mô tả	Loại Rendering	Yêu cầu Auth
/profile	Xem và chỉnh sửa thông tin cá nhân	CSR	Bắt buộc
/search	Trang kết quả full-text search	SSR	Không

5.2. Giao diện Phần mềm – REST API

Backend cung cấp RESTful API theo chuẩn JSON. Toàn bộ endpoints được tiền tố /api/v1. Xem chi tiết tại Chương 8.

Giao thức	HTTP/1.1 và HTTP/2 qua HTTPS (TLS 1.2+). Nginx termination SSL.
Base URL (dev)	http://localhost:5000/api/v1
Base URL (prod)	https://api.culinaryblog.com/api/v1
Content-Type	application/json; charset=utf-8 (request và response). Multipart/form-data cho file upload endpoints.
Authentication	Bearer Token trong Authorization header: Authorization: Bearer <access_token>. Refresh token: trong request body (không dùng cookie để tránh CSRF).
Response Format	Success: { "data": {...}, "meta": { "page": 1, "pageSize": 10, "total": 100 } } Error: RFC 7807 Problem Details { "type", "title", "status", "detail", "errors": {} }
Versioning	URL Path versioning: /api/v1/. Khi có breaking changes → /api/v2/ (v1 được duy trì tối thiểu 6 tháng).
CORS Headers	Access-Control-Allow-Origin: <configured-origins> Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE, OPTIONS Access-Control-Allow-Headers: Content-Type, Authorization, X-Correlation-ID
Rate Limit Headers	X-RateLimit-Limit: 100 X-RateLimit-Remaining: 87 X-RateLimit-Reset: 1700000000 (Unix timestamp) Retry-After: 30 (seconds, khi 429)
Correlation ID	X-Correlation-ID header: sinh tự động nếu không có trong request, trả về trong response. Gán vào tất cả log entries (Serilog MDC).

5.3. Giao diện Dịch vụ Bên thứ ba

Dịch vụ	Mục đích	Giao thức / SDK	Cấu hình / Secrets
Google OAuth 2.0	Đăng nhập / đăng ký bằng tài	OAuth 2.0 Authorization Code + PKCE. Redirect URI: /api/v1/auth/google/callback. Scopes: openid, email, profile.	GoogleClientId, GoogleClientSecret (User Secrets / env var). Console → OAuth 2.0 Client

Dịch vụ	Mục đích	Giao thức / SDK	Cấu hình / Secrets
	khoản Google		
MinIO (S3-compatible)	Lưu trữ file ảnh công thức	AWS SDK for .NET (AWSSDK.S3). Endpoint override cho MinIO. Presigned URL cho direct browser upload (optional).	MinIO__Endpoint, MinIO__MinIO__SecretKey, MinIO__BucketName. Dev: minio:9000.
Hangfire	Background job processing	Nuget: Hangfire.Core, Hangfire.AspNetCore, Hangfire.PostgreSql. In-process server. Dashboard: /hangfire (Admin only, policy-protected).	Dùng chung ConnectionString PostgreSQL. HANGFIRE__hangfire.
Serilog + Seq	Structured logging & log aggregation	Serilog.Sinks.Console (JSON), Serilog.Sinks.File, Serilog.Sinks.Seq. HTTP ingest API.	Seq__ServerUrl = http://seq (Docker). Production: Elasticsearch Monitor.
OpenTelemetry	Distributed tracing & metrics	OpenTelemetry .NET SDK. OTLP exporter. Tracing: HttpClient, EF Core, ASP.NET Core.	OTEL__EXPORTER_OTLP Development: Seq OTLP. Grafana Tempo / Jaeger.
SMTP / Email	Gửi welcome email (FR-JOB-001)	MailKit (IEmailSender). Kết nối qua SMTP với TLS.	Smtp__Host, Smtp__Port, Smtp__Username, Smtp__Development: Mailhog (Docker).
Google Search Console	Ping sitemap update	HTTP GET: https://www.google.com/ping?sitemap={url}	Không cần API key. Gọi tr 003.

5.4. Giao diện Phần cứng

Hệ thống là web application, không giao tiếp trực tiếp với phần cứng chuyên biệt. Yêu cầu phần cứng tối thiểu cho server:

Thành phần	Development (local)	Production (minimum)
CPU	2 cores (Intel/AMD/ARM64 — Apple M-series được hỗ trợ qua Docker)	2 vCPU (VPS/Cloud instance, x86_64)
RAM	8 GB (chạy Docker Compose đầy đủ: API + PG + Redis + MinIO + Seq)	4 GB (API + dependencies riêng lẻ)
Storage	20 GB SSD (cho Docker images + database data + MinIO volumes)	50 GB SSD (production data growth)
Network	Kết nối internet (npm/nuget packages, Google OAuth)	Bandwidth ≥ 1 Gbps, IP tĩnh

Thành phần	Development (local)	Production (minimum)
Browser Client	Chrome 112+, Firefox 113+, Safari 16+, Edge 112+ (ES2020+)	Tương tự — không hỗ trợ IE11

6. Kiến trúc Hệ thống

Chương này mô tả tổng quan kiến trúc phần mềm của hệ thống Culinary Blog. Hệ thống được thiết kế theo mô hình Client-Server với hai tầng riêng biệt: Frontend (Next.js) và Backend (.NET 10 Minimal API), giao tiếp qua REST API. Backend tuân thủ nguyên tắc Clean Architecture kết hợp CQRS pattern.

6.1. Tổng quan Kiến trúc

Tầng	Technology	Vai trò	Giao tiếp với
Client (Browser/Mobile)	Browser (Chrome/Firefox/Safari)	Người dùng tương tác qua giao diện web	Next.js App
Frontend	Next.js 14+ App Router, TypeScript, Tailwind CSS, Auth.js v5, TanStack Query, React Hook Form + Zod	Rendering UI, route management, client-side state. SSR/ISR cho SEO.	Backend REST API
Nginx Reverse Proxy	Nginx Alpine (Docker)	SSL termination, load balancing, static file caching, rate limiting basic.	Frontend :3000, Backend API :5000
Backend API	ASP.NET Core .NET 10 Minimal API	Business logic, authentication, data access, background jobs.	PostgreSQL, Redis, MinIO, Email
Cache Layer	Redis 7	Distributed cache cho recipe/category/search results. Rate limiting counters.	Backend API
Object Storage	MinIO (S3-compatible)	Lưu file ảnh: original, medium (800×600), thumbnail (300×300).	Backend API (via AWSSDK.S3)
Database	PostgreSQL 16	Persistent relational data storage. Full-text search via tsvector.	Backend API (via EF Core)
Observability	Serilog + Seq, OpenTelemetry + Grafana/Jaeger	Logging, metrics, distributed tracing.	Backend API

6.2. Kiến trúc Backend – Clean Architecture

Backend tuân thủ Clean Architecture (Robert C. Martin) với nguyên tắc Dependency Rule: dependency chỉ đi vào trong (hướng Domain). Không bao giờ có reference từ Domain/Application ra Infrastructure.

Domain Layer (CulinaryBlog.Domain)	Nhân lõi hệ thống. Chứa: • Entities: Recipe, Category, ApplicationUser, RecipeStep, RecipeIngredient, RecipeImage. • Value Objects: Slug, EmailAddress. • Owned Entities: RecipeNutrition. • Domain Events (optional): RecipePublishedEvent. • Enums: RecipeDifficulty, RecipeStatus. • Interfaces: IRepository<T>, IRecipeRepository, ICategoryRepository. • Không có NuGet dependencies (chỉ .NET BCL).
Application Layer (CulinaryBlog.Application)	Orchestration Layer. Chứa: • Commands (CQRS write): CreateRecipeCommand, PublishRecipeCommand, LoginCommand... • Queries (CQRS read): GetRecipesQuery, GetRecipeBySlugQuery... • Handlers (MediatR IRequestHandler): xử lý logic business cho mỗi command/query. • DTOs / Response models: RecipeDto, UserDto, PagedResult<T>. • Validators (FluentValidation): validation rules cho mỗi command. • Pipeline Behaviors: ValidationBehavior, LoggingBehavior, CachingBehavior, PerformanceBehavior. • Service interfaces: IEmailService, IJwtService, IFileStorageService, ICurrentUser.
Infrastructure Layer (CulinaryBlog.Infrastructure)	Implements application interfaces. Chứa: • EF Core: CulinaryBlogDbContext, configurations, migrations, repositories. • Repository implementations: RecipeRepository (LINQ + EF Core + FTS), CategoryRepository. • JWT Service: JwtService (System.IdentityModel.Tokens.Jwt). • File Storage: MinioFileStorageService (AWSSDK.S3). • Email: MailKitEmailService. • Cache: RedisCacheService (StackExchange.Redis). • Hangfire job registrations. • EF Core Interceptors: AuditInterceptor (auto set CreatedAt/UpdatedAt).
Presentation Layer (CulinaryBlog.API)	HTTP interface. Chứa: • Minimal API Endpoint Groups: AuthEndpoints, RecipesEndpoints, CategoriesEndpoints. • Middleware: GlobalExceptionHandler, CorrelationIdMiddleware, RateLimitingMiddleware. • DI Configuration: Program.cs + Extension methods (AddApplication, AddInfrastructure, AddPresentation). • OpenAPI: Scalar UI tại /scalar, XML documentation comments. • Authentication: JWT Bearer + Google OAuth via Auth.js v5 (frontend) hoặc ASP.NET Google provider.

6.3. CQRS + MediatR Pipeline

CQRS (Command Query Responsibility Segregation) tách biệt read và write models. Mỗi request đi qua MediatR Pipeline Behaviors theo thứ tự:

Thứ tự	Pipeline Behavior	Trách nhiệm	Áp dụng cho
1	LoggingBehavior	Log request type, parameters, elapsed time. Cảnh báo nếu > 500ms.	Tất cả Commands và Queries
2	ValidationBehavior	Chạy FluentValidation validators đã đăng ký. Throw ValidationException nếu có lỗi.	Tất cả Commands và Queries có Validator
3	CachingBehavior	Kiểm tra Redis cache trước khi xử lý. Implements ICacheable interface trên Query.	Queries implements ICacheable (GET endpoints)
4	Handler (IRequestHandler)	Thực thi business logic: gọi repositories, raise domain events, tạo response DTO.	Tất cả (bắt buộc)
5	CacheInvalidationBehavior	Xóa cache liên quan sau khi Command thành công. Implements ICacheInvalidator.	Commands thay đổi data (Create/Update/Delete)

6.4. Mô hình Quan hệ Thực thể (ERD tóm tắt)

Hệ thống sử dụng PostgreSQL 16 với EF Core Code First. Tất cả entities kế thừa BaseEntity (Id, CreatedAt, UpdatedAt, IsDeleted, RowVersion).

Thực thể	Quan hệ	Bảng PostgreSQL
Recipe	Nhiều RecipeStep (1:N) Nhiều RecipeIngredient (1:N) Nhiều RecipeImage (1:N) Một RecipeNutrition (1:1 Owned) Một Category (N:1) Một Author/ApplicationUser (N:1)	"Recipes" "RecipeSteps" "RecipeIngredients" "RecipeImages" (owned — cột trong Recipes) "Categories" "AspNetUsers"
ApplicationUser	Nhiều Recipe (Author, 1:N) Nhiều RefreshToken (1:N)	"AspNetUsers" (Identity) "RefreshTokens"
Category	Nhiều Recipe (1:N)	"Categories"
RefreshToken	Một ApplicationUser (N:1)	"RefreshTokens"

6.5. Triển khai – Docker Compose

Toàn bộ hệ thống được containerized với Docker Compose. Development dùng docker-compose.yml, Production dùng docker-compose.prod.yml với optimized build + secrets management.

Service	Image	Port (host:container)	Volume / Dependency
nginx	nginx:alpine	80:80, 443:443	Depends: api, frontend Volume: ./nginx/nginx.conf, ./ssl/
api	culinaryblog-api (Dockerfile)	5000:8080	Depends: postgres, redis, minio Env file: .env.production
frontend	culinaryblog-web (Dockerfile)	3000:3000	Depends: api
postgres	postgres:16-alpine	5432:5432	Volume: pgdata:/var/lib/postgresql/data Env: POSTGRES_DB, USER, PASSWORD
redis	redis:7-alpine	6379:6379	Volume: redisdata:/data Command: redis-server --appendonly yes
minio	minio/minio:latest	9000:9000, 9001:9001 (Console)	Volume: miniodata:/data Command: server /data --console-address :9001
seq	datalust/seq:latest	5341:80	Volume: seqdata:/data Dev only — không deploy production
mailhog	mailhog/mailhog	8025:8025 (UI), 1025:1025 (SMTP)	Dev only — test email

7. Mô hình Dữ liệu

Chương này đặc tả cấu trúc dữ liệu đầy đủ của hệ thống Culinary Blog. Tất cả entities kế thừa BaseEntity và sử dụng Soft Delete pattern (IsDeleted flag). Database: PostgreSQL 16 với EF Core 10 Code First.

7.1. BaseEntity (Abstract)

Tất cả thực thể kế thừa từ BaseEntity. Không tạo bảng riêng (Table-Per-Hierarchy không được dùng ở đây — mỗi entity có bảng riêng với các cột kế thừa).

Column	Kiểu dữ liệu	Ràng buộc	Mô tả
Id	uuid (Guid)	PRIMARY KEY, DEFAULT gen_random_uuid()	Khóa chính UUID v4 — tránh sequential ID guessing.
CreatedAt	timestampz	NOT NULL, DEFAULT NOW()	Thời điểm tạo bản ghi. Set bởi AuditInterceptor (EF Core).
UpdatedAt	timestampz	NULL	Thời điểm cập nhật cuối. Set bởi AuditInterceptor khi SaveChanges.
IsDeleted	boolean	NOT NULL, DEFAULT false	Soft delete flag. Global Query Filter: .Where(x => !x.IsDeleted).
RowVersion	bytea (timestamp)	NOT NULL, Concurrency Token	Optimistic concurrency control. EF Core [Timestamp] annotation.

7.2. Recipe

Thực thể trung tâm của hệ thống. Một Recipe thuộc một Category và một Author. Chứa Owned Entity RecipeNutrition và các Collection Navigation Properties.

Column	Kiểu dữ liệu	Ràng buộc	Index	Mô tả
Id	uuid	PK (kế thừa)	PK	(BaseEntity)
Title	varchar(200)	NOT NULL	IDX_Recipe_Title (GIN trigram — optional)	Tiêu đề công thức. Unique không bắt buộc (có thể trùng title khác nhau slug).
Slug	varchar(220)	NOT NULL, UNIQUE	IDX_Recipe_Slug (UNIQUE B-tree)	URL-friendly identifier. Sinh

Column	Kiểu dữ liệu	Ràng buộc	Index	Mô tả
				từ Title + chuẩn hóa (lowercase, replace space → -). Không thay đổi sau Publish.
Description	text	NOT NULL	—	Mô tả ngắn (≤ 2000 ký tự). Hiển thị trong card preview và SEO meta description.
Instructions	text	NOT NULL	—	Hướng dẫn tổng quan dạng markdown (legacy field). Chi tiết dùng RecipeSteps.
PrepTime	integer	NOT NULL, CHECK > 0	—	Thời gian chuẩn bị (phút).
CookTime	integer	NOT NULL, CHECK ≥ 0	—	Thời gian nấu (phút). 0 cho "No cook" recipes.
Servings	integer	NOT NULL, CHECK > 0	—	Số khẩu phần (portions).
Difficulty	smallint (enum)	NOT NULL, DEFAULT 1	IDX_Recipe_Difficulty	RecipeDifficulty: 1=Easy, 2=Medium, 3=Hard, 4=Expert.
Status	smallint (enum)	NOT NULL, DEFAULT 0	IDX_Recipe_Status	RecipeStatus: 0=Draft, 1=Published, 2=Archived.
CategoryId	uuid	NOT NULL, FK → Categories.Id	IDX_Recipe_CategoryId (B-tree)	Khóa ngoại đến Category. ON DELETE RESTRICT (không xóa category có recipe).
AuthorId	varchar(450)	NOT NULL, FK → AspNetUsers.Id	IDX_Recipe_AuthorId (B-tree)	Khóa ngoại đến ApplicationUser (Author).

Column	Kiểu dữ liệu	Ràng buộc	Index	Mô tả
SearchVector	tsvector	NULL	IDX_Recipe_Search (GIN)	Full-text search vector. Được cập nhật bởi PostgreSQL TRIGGER khi Title/Description thay đổi. Dùng unaccent extension cho tiếng Việt.
PublishedAt	timestampz	NULL	IDX_Recipe_PublishedAt	Thời điểm publish. Set khi Status chuyển sang Published. NULL nếu chưa publish.
CreatedAt	timestampz	NOT NULL	—	(BaseEntity)
UpdatedAt	timestampz	NULL	—	(BaseEntity)
IsDeleted	boolean	NOT NULL	IDX_Recipe_IsDeleted (partial)	(BaseEntity) — Global Query Filter.
RowVersion	bytea	NOT NULL	—	(BaseEntity) — Optimistic concurrency.

7.2.1. RecipeNutrition (Owned Entity — cột trong bảng Recipes)

Owned Entity — không có bảng riêng. Các cột được nhúng trực tiếp vào bảng Recipes với tiền tố "Nutrition_".

Column trong DB	Property C#	Kiểu	Mô tả
Nutrition_Calories	Calories	decimal(8,2)?	Năng lượng (kcal / serving). Nullable.
Nutrition_Protein	Protein	decimal(8,2)?	Đạm (gram / serving). Nullable.
Nutrition_Carbohydrates	Carbohydrates	decimal(8,2)?	Tinh bột (gram / serving). Nullable.
Nutrition_Fat	Fat	decimal(8,2)?	Chất béo (gram / serving). Nullable.
Nutrition_Fiber	Fiber	decimal(8,2)?	Chất xơ (gram / serving). Nullable.
Nutrition_Sodium	Sodium	decimal(8,2)?	Natri (mg / serving). Nullable.

7.3. RecipeStep

Các bước thực hiện chi tiết của một Recipe, được sắp xếp theo StepNumber.

Column	Kiểu	Ràng buộc	Mô tả
Id	uuid	PK (BaseEntity)	UUID khóa chính.
RecipeId	uuid	NOT NULL, FK → Recipes.Id, ON DELETE CASCADE	Khóa ngoại. Cascade delete: xóa Recipe → xóa tất cả Steps.
StepNumber	integer	NOT NULL, CHECK > 0	Thứ tự bước (1, 2, 3...). UNIQUE cùng RecipeId (composite unique).
Title	varchar(200)	NOT NULL	Tên bước ngắn gọn (ví dụ: "Sơ chế nguyên liệu").
Description	text	NOT NULL	Mô tả chi tiết bước thực hiện.
TimerMinutes	integer	NULL, CHECK >= 0	Thời gian cần cho bước này (phút). NULL nếu không áp dụng.
ImageUrl	varchar(500)	NULL	URL ảnh minh họa bước (trên MinIO). Nullable.

7.4. RecipeIngredient

Column	Kiểu	Ràng buộc	Mô tả
Id	uuid	PK (BaseEntity)	UUID khóa chính.
RecipeId	uuid	NOT NULL, FK → Recipes.Id, ON DELETE CASCADE	Khóa ngoại với cascade delete.
Name	varchar(200)	NOT NULL	Tên nguyên liệu (ví dụ: "Thịt bò thăn").
Quantity	decimal(10,3)	NULL	Số lượng (ví dụ: 500). Nullable cho "nguyên liệu vừa đủ".
Unit	varchar(50)	NULL	Đơn vị đo lường (gram, ml, thìa canh, quả...). Nullable.
Notes	varchar(500)	NULL	Ghi chú tùy chọn (ví dụ: "thái lát mỏng"). Nullable.
OrderIndex	integer	NOT NULL, DEFAULT 0	Thứ tự hiển thị trong danh sách nguyên liệu.

7.5. RecipeImage

Column	Kiểu	Ràng buộc	Mô tả
Id	uuid	PK (BaseEntity)	UUID khóa chính.
RecipeId	uuid	NOT NULL, FK → Recipes.Id, ON DELETE CASCADE	Khóa ngoại với cascade delete.
OriginalUrl	varchar(500)	NOT NULL	URL ảnh gốc trên MinIO (ví dụ: ../recipes/{recipeId}/{guid}.jpg).
MediumUrl	varchar(500)	NULL	URL ảnh medium 800×600 (sinh bởi FR-JOB-002). Nullable khi job chưa chạy.
ThumbnailUrl	varchar(500)	NULL	URL ảnh thumbnail 300×300 (sinh bởi FR-JOB-002). Nullable.
AltText	varchar(200)	NULL	Alt text cho accessibility. Nullable.
IsPrimary	boolean	NOT NULL, DEFAULT false	Ảnh chính (hiển thị đầu tiên). Chỉ có 1 ảnh IsPrimary=true / Recipe.
OrderIndex	integer	NOT NULL, DEFAULT 0	Thứ tự hiển thị gallery.

7.6. Category

Column	Kiểu	Ràng buộc	Mô tả
Id	uuid	PK (BaseEntity)	UUID khóa chính.
Name	varchar(100)	NOT NULL, UNIQUE	Tên danh mục (ví dụ: "Món khai vị").
Slug	varchar(120)	NOT NULL, UNIQUE, IDX_Category_Slug	URL-friendly name. Sinh từ Name.
Description	text	NULL	Mô tả danh mục. Nullable.
ImageUrl	varchar(500)	NULL	URL ảnh đại diện category. Nullable.
OrderIndex	integer	NOT NULL, DEFAULT 0	Thứ tự hiển thị trên navigation.

7.7. ApplicationUser (extends IdentityUser)

Kế thừa từ ASP.NET Core Identity IdentityUser<string>. Bảng: "AspNetUsers". Thêm các custom columns:

Column (custom)	Kiểu	Ràng buộc	Mô tả
DisplayName	varchar(100)	NOT NULL	Tên hiển thị công khai (không phải username).
AvatarUrl	varchar(500)	NULL	URL ảnh avatar. Nullable. Sinh từ Google Avatar khi đăng ký OAuth.
Bio	text	NULL	Tiểu sử ngắn của tác giả. Nullable. Hiển thị trên author profile.
IsActive	boolean	NOT NULL, DEFAULT true	Trạng thái tài khoản. Admin có thể deactivate user (ban).
CreatedAt	timestampz	NOT NULL, DEFAULT NOW()	Ngày tạo tài khoản.

Identity columns (kế thừa): Id (varchar 450), UserName, NormalizedUserName, Email, NormalizedEmail, PasswordHash, SecurityStamp, ConcurrencyStamp, PhoneNumber, TwoFactorEnabled, LockoutEnd, LockoutEnabled, AccessFailedCount.

7.8. RefreshToken

Column	Kiểu	Ràng buộc	Mô tả
Id	uuid	PK	UUID khóa chính.
UserId	varchar(450)	NOT NULL, FK →AspNetUsers.Id, ON DELETE CASCADE	Chủ sở hữu token.
TokenHash	varchar(64)	NOT NULL, UNIQUE, IDX_RefreshToken_Hash	SHA-256 hash của raw token. Không lưu raw token.
ExpiresAt	timestampz	NOT NULL	Thời hạn token (7 ngày kể từ CreatedAt).
RevokedAt	timestampz	NULL	Thời điểm revoke. NULL = còn hiệu lực.
ReplacedByTokenHash	varchar(64)	NULL	Hash của token mới (khi rotation). Để trace token family.
CreatedAt	timestampz	NOT NULL, DEFAULT NOW()	Thời điểm tạo.

Column	Kiểu	Ràng buộc	Mô tả
CreatedByIp	varchar(45)	NULL	IP address tạo token. Lưu để audit.

8. Đặc tả REST API

Chương này liệt kê tất cả API endpoints của hệ thống Culinary Blog. Base URL: /api/v1. Tài liệu chi tiết (request/response schemas) được sinh tự động qua Scalar UI tại /scalar.

Convention	HTTP Method + Path (prefixed /api/v1) auth required = Bearer JWT Access Token bắt buộc role = Role tối thiểu cần thiết (Author $\subset$ Admin)
Pagination	Query params: ?page=1&pageSize=10&sortBy=createdAt&sortOrder=desc Response wrapper: { "data":[], "meta":{"page", "pageSize", "total", "totalPages" } }
Error Format	RFC 7807 Problem Details: { "type":"about:blank", "title":"...", "status":400, "detail":"...", "errors":{"field":["msg"]} }

8.1. Authentication Module (/auth)

Method	Endpoint	Mô tả	Auth	Request Body / Params	Response
POST	/auth/register	Đăng ký tài khoản mới	Không	{ email, password, displayName }	201: { userId, email, displayName } 400: validation errors 409: email đã tồn tại
POST	/auth/login	Đăng nhập email/password	Không	{ email, password }	200: { accessToken, refreshToken, expiresIn } 401: sai credentials 429: quá giới hạn rate limit
POST	/auth/google	Đăng nhập Google OAuth	Không	{ idToken } — ID Token từ Google Sign-In JS SDK	200: { accessToken, refreshToken, expiresIn } 400: invalid token
POST	/auth/refresh	Làm mới Access Token	Không (dùng refreshToken)	{ refreshToken }	200: { accessToken, refreshToken, expiresIn }

Method	Endpoint	Mô tả	Auth	Request Body / Params	Response
					expiresIn } 401: token hết hạn / bị revoke
POST	/auth/logout	Đăng xuất, revoke Refresh Token	Bearer JWT	{ refreshToken }	204: No Content 401: Unauthorized
GET	/auth/me	Lấy thông tin user hiện tại	Bearer JWT	—	200: { id, email, displayName, avatarUrl, bio, roles } 401: Unauthorized
PATCH	/auth/me	Cập nhật profile người dùng	Bearer JWT	{ displayName?, avatarUrl?, bio? }	200: { id, email, displayName, avatarUrl, bio } 400: validation 401: Unauthorized

8.2. Categories Module (/categories)

Method	Endpoint	Mô tả	Auth / Role	Request	Response
GET	/categories	Lấy danh sách tất cả categories	Không	—	200: [{ id, name, slug, description, imageUrl, recipeCount }] 400: validation
GET	/categories/{slug}	Lấy chi tiết category + danh sách recipes	Không	?page=1&pageSize=10&sortBy=...	200: { category, recipes: PagedResult } 404: Category not found
POST	/categories	Tạo category mới	Bearer + Admin	{ name, description?, imageUrl? }	201: { id, name, slug, description } 400: validation

Method	Endpoint	Mô tả	Auth / Role	Request	Response
					validation 403: Forbidden 409: name đã tồn tại
PUT	/categories/{id}	Cập nhật category	Bearer + Admin	{ name, description?, imageUrl?, orderIndex? }	200: category updated 400/403/404
DELETE	/categories/{id}	Xóa category (soft delete)	Bearer + Admin	—	204: No Content 403: Forbidden 404: Not found 409: Có recipes thuộc category này

8.3. Recipes Module (/recipes)

Method	Endpoint	Mô tả	Auth / Role	Request
GET	/recipes	Danh sách recipes (Published, paginated)	Không	?page&pageSize&sortBy&sortOrder&
GET	/recipes/{slug}	Chi tiết recipe theo slug (kèm steps, ingredients, images, nutrition)	Không (Draft: Author/Admin)	—
GET	/recipes/search	Full-text search công thức	Không	?q={keyword}&page&pageSize&cate
POST	/recipes	Tạo recipe mới (trạng thái Draft)	Bearer (Author/Admin)	{ title, description, categoryId, prepTi nutrition? }
PUT	/recipes/{id}	Cập nhật thông tin cơ bản recipe	Bearer (Owner/Admin)	{ title?, description?, categoryId?, pre instructions?, nutrition? }
PATCH	/recipes/{id}/publish	Publish recipe	Bearer (Owner/Admin)	—

Method	Endpoint	Mô tả	Auth / Role	Request
		(Draft → Published)		
PATCH	/recipes/{id}/unpublish	Unpublish recipe (Published → Draft)	Bearer (Owner/Admin)	—
PATCH	/recipes/{id}/archive	Archive recipe	Bearer (Owner/Admin)	—
DELETE	/recipes/{id}	Xóa recipe (soft delete)	Bearer (Owner/Admin)	—

8.4. Recipe Images (/recipes/{id}/images)

Method	Endpoint	Mô tả	Auth	Request	Response
POST	/recipes/{id}/images	Upload ảnh mới cho recipe	Bearer (Owner/Admin)	multipart/form-data: file (image), altText?, isPrimary?	201: { imageId, originalUrl, altText, isPrimary } 400: MIME invalid / size > 5MB 403/404
PATCH	/recipes/{id}/images/{imageId}	Cập nhật metadata ảnh (altText, isPrimary, orderIndex)	Bearer (Owner/Admin)	{ altText?, isPrimary?, orderIndex? }	200: image updated 403/404
DELETE	/recipes/{id}/images/{imageId}	Xóa ảnh (MinIO file deleted async via Hangfire)	Bearer (Owner/Admin)	—	204: No Content 403/404

8.5. Recipe Steps (/recipes/{id}/steps)

Method	Endpoint	Mô tả	Auth	Request	Response
POST	/recipes/{id}/steps	Thêm bước mới vào recipe	Bearer (Owner/Admin)	{ stepNumber, title, description, timerMinutes?, imageUrl? }	201: RecipeStepDto 400/403/404
PUT	/recipes/{id}/steps/{stepId}	Cập nhật một bước	Bearer (Owner/Admin)	{ stepNumber?, title?, description?, timerMinutes?, imageUrl? }	200: RecipeStepDto 400/403/404
DELETE	/recipes/{id}/steps/{stepId}	Xóa một bước	Bearer (Owner/Admin)	—	204: No Content 403/404

8.6. Recipe Ingredients (/recipes/{id}/ingredients)

Method	Endpoint	Mô tả	Auth	Request	Response
POST	/recipes/{id}/ingredients	Thêm nguyên liệu	Bearer (Owner/Admin)	{ name, quantity?, unit?, notes?, orderIndex? }	201: RecipeIngredient 400/403/404
PUT	/recipes/{id}/ingredients/{ingId}	Cập nhật nguyên liệu	Bearer (Owner/Admin)	{ name?, quantity?, unit?, notes?, orderIndex? }	200: RecipeIngredient 400/403/404
DELETE	/recipes/{id}/ingredients/{ingId}	Xóa nguyên liệu	Bearer (Owner/Admin)	—	204: No Content 403/404

8.7. Health Check Endpoints

Method	Endpoint	Mô tả	Auth	Response
GET	/health	Tổng hợp health tất cả dependencies (DB, Redis, MinIO)	Không	200: Healthy 503: Unhealthy { "status":"Healthy", "entries":{"database":{"status":"Healthy"},...} }

Method	Endpoint	Mô tả	Auth	Response
GET	/health/live	Liveness probe — chỉ kiểm tra process còn sống	Không	200: Healthy (luôn luôn, trừ khi process crashed)
GET	/health/ready	Readiness probe — kiểm tra DB và Redis sẵn sàng	Không	200: Healthy (DB + Redis up) 503: Unhealthy (không nhận traffic)

Phụ lục A – HTTP Status Codes

Bảng dưới đây liệt kê tất cả HTTP Status Codes được sử dụng trong API Culinary Blog, cùng ngữ cảnh sử dụng cụ thể.

Code	Status	Ngữ cảnh sử dụng
200	OK	GET request thành công; PATCH trả về resource đã cập nhật; POST /auth/login thành công.
201	Created	POST tạo resource mới thành công (Recipe, Category, Step, Ingredient, Image). Response body chứa resource vừa tạo.
204	No Content	DELETE thành công; POST /auth/logout thành công. Không có response body.
400	Bad Request	Validation lỗi (FluentValidation), request body malformed, file MIME không hợp lệ, business rule vi phạm (ví dụ: publish recipe thiếu ingredients).
401	Unauthorized	Access Token thiếu hoặc invalid; Refresh Token hết hạn / bị revoke.
403	Forbidden	Đã xác thực nhưng không có quyền: Author truy cập endpoint Admin; Author cố xóa recipe của người khác.
404	Not Found	Resource không tồn tại hoặc đã soft-delete (IsDeleted=true).
409	Conflict	Trùng lặp unique field (email đã đăng ký, category slug đã tồn tại); Xóa category đang có recipes.
422	Unprocessable Entity	Dữ liệu hợp lệ về cú pháp nhưng không thể xử lý về ngữ nghĩa (ví dụ: RowVersion conflict — Optimistic Concurrency).
429	Too Many Requests	Rate limit bị vượt. Response kèm header Retry-After (giây).
500	Internal Server Error	Lỗi không xử lý được (unhandled exception). Trả RFC 7807, log đầy đủ qua Serilog. Không lộ stack trace.
503	Service Unavailable	Health check failed (DB/Redis down); hoặc server overloaded.

Phụ lục B – Application Error Codes

Hệ thống sử dụng Application Error Codes (mã lỗi tùy chỉnh) trong trường RFC 7807 "type" để frontend có thể xử lý lỗi theo programmatic way mà không phụ thuộc vào chuỗi message (có thể thay đổi theo locale).

Error Code	HTTP Status	Mô tả	Module
AUTH_EMAIL_EXISTS	409	Email đã được đăng ký bởi tài khoản khác.	Auth
AUTH_INVALID_CREDENTIALS	401	Email hoặc mật khẩu không đúng.	Auth
AUTH_TOKEN_EXPIRED	401	Access Token đã hết hạn (15 phút).	Auth
AUTH_TOKEN_INVALID	401	Access Token sai định dạng hoặc chữ ký không hợp lệ.	Auth
AUTH_REFRESH_TOKEN_EXPIRED	401	Refresh Token đã hết hạn (7 ngày).	Auth
AUTH_REFRESH_TOKEN_REVOKED	401	Refresh Token đã bị thu hồi (reuse detection).	Auth
AUTH_GOOGLE_TOKEN_INVALID	400	Google ID Token không hợp lệ hoặc đã hết hạn.	Auth
AUTH_ACCOUNT_DISABLED	403	Tài khoản bị vô hiệu hóa (IsActive=false) bởi Admin.	Auth
RECIPE_NOT_FOUND	404	Recipe với id/slug không tồn tại hoặc đã bị xóa.	Recipe
RECIPE_SLUG_EXISTS	409	Slug đã tồn tại — tự động thêm suffix (slug-1, slug-2...).	Recipe
RECIPE_PUBLISH_INCOMPLETE	400	Recipe thiếu điều kiện publish: phải có ít nhất 1 ingredient và 1 step.	Recipe
RECIPE_FORBIDDEN	403	User không phải owner và không phải Admin.	Recipe
RECIPE_CONCURRENCY_CONFLICT	422	RowVersion không khớp — resource đã được cập nhật bởi request khác. Client cần reload.	Recipe
CATEGORY_NOT_FOUND	404	Category không tồn tại.	Category
CATEGORY_NAME_EXISTS	409	Tên category đã tồn tại.	Category

Error Code	HTTP Status	Mô tả	Module
CATEGORY_DELETE_HAS_RECIPES	409	Không thể xóa category đang có recipes thuộc về.	Category
FILE_SIZE_EXCEEDED	400	File upload vượt quá giới hạn 5MB.	File
FILE_MIME_INVALID	400	Loại file không được phép. Chỉ chấp nhận JPEG, PNG, WebP, AVIF.	File
VALIDATION_ERROR	400	Một hoặc nhiều field không hợp lệ. Xem "errors" object.	Common
RATE_LIMIT_EXCEEDED	429	Quá giới hạn request. Xem Retry-After header.	Common

Phụ lục C – Từ điển Thuật ngữ

Thuật ngữ	Viết tắt	Định nghĩa
Access Token	AT	JSON Web Token (JWT) dùng để xác thực API request. TTL = 15 phút. Ký bằng HS256.
Application Error Code	AEC	Mã lỗi tùy chỉnh dạng SCREAMING_SNAKE_CASE trong trường "type" của RFC 7807 Problem Details.
Archive	—	Trạng thái Recipe khi bị ẩn khỏi public listing nhưng không bị xóa. RecipeStatus.Archived.
Author	—	Role người dùng mặc định sau khi đăng ký. Có thể tạo/quản lý recipe của mình.
Background Job	—	Tác vụ xử lý bất đồng bộ chạy ngoài HTTP request cycle, quản lý bởi Hangfire.
Clean Architecture	CA	Kiến trúc phần mềm của Robert C. Martin tách biệt concerns theo layers (Domain, Application, Infrastructure, Presentation). Dependency chỉ đi vào trong (hướng Domain).
Command Query Responsibility Segregation	CQRS	Pattern tách biệt write model (Commands) và read model (Queries) để tối ưu từng luồng riêng.
Content Delivery Network	CDN	Mạng phân phối nội dung tĩnh (ảnh, JS, CSS) từ server gần người dùng nhất.

Thuật ngữ	Viết tắt	Định nghĩa
Core Web Vitals	CWV	Chỉ số đo lường UX của Google: LCP (tải trang), CLS (ổn định layout), INP (phản hồi tương tác).
CQRS	-	Xem Command Query Responsibility Segregation
Docker Compose	—	Công cụ định nghĩa và chạy multi-container Docker application qua file YAML.
Draft	—	Trạng thái mặc định của Recipe khi mới tạo. Chỉ Author/Admin thấy.
Full-Text Search	FTS	Tìm kiếm ngôn ngữ tự nhiên trong PostgreSQL qua tsvector/tsquery + unaccent extension.
Hangfire	—	Thư viện .NET xử lý background jobs: fire-and-forget, delayed, recurring.
HTTP Status Code	—	Mã phản hồi HTTP chuẩn (RFC 7231) cho biết kết quả xử lý request (2xx: thành công, 4xx: client error, 5xx: server error).
Incremental Static Regeneration	ISR	Tính năng Next.js tái sinh (regenerate) trang tĩnh theo chu kỳ (revalidate interval) thay vì build lại toàn bộ.
JSON Web Token	JWT	Chuẩn mở (RFC 7519) định nghĩa cách truyền thông tin an toàn giữa các bên dưới dạng JSON object được ký.
MediatR	—	Thư viện .NET triển khai Mediator pattern. Dispatch Commands/Queries qua Handler có pipeline behaviors.
MinIO	—	Object storage server mã nguồn mở tương thích Amazon S3 API. Dùng để lưu trữ ảnh.
Non-Functional Requirement	NFR	Yêu cầu chất lượng hệ thống: hiệu năng, bảo mật, độ tin cậy, khả năng bảo trì...
Nginx	—	Web server hiệu năng cao, dùng làm reverse proxy, load balancer và SSL termination.
OpenTelemetry	OTEL	Framework quan sát hệ thống phân tán: distributed tracing, metrics, logs.
Optimistic Concurrency	—	Kỹ thuật xử lý concurrent writes bằng RowVersion — không lock DB, phát hiện conflict khi save.
Published	—	Trạng thái Recipe khi được công bố công khai. RecipeStatus.Published.
Rate Limiting	—	Giới hạn số lượng request từ một IP trong khoảng thời gian nhất định để ngăn brute force/DDoS.
Refresh Token	RT	Token dài hạn (7 ngày) dùng để lấy Access Token mới mà không cần đăng nhập lại.
Refresh Token Rotation	—	Mỗi lần dùng Refresh Token để refresh → token cũ bị revoke, cấp token mới (bảo mật cao hơn).

Thuật ngữ	Viết tắt	Định nghĩa
Reuse Detection	—	Cơ chế phát hiện khi Refresh Token đã bị revoke được dùng lại → revoke toàn bộ token family của user.
Slug	—	Chuỗi URL-friendly, dạng chữ-thường-gạch-nối, duy nhất, dùng để định danh Recipe/Category trên URL.
Soft Delete	—	Đánh dấu IsDeleted=true thay vì xóa vật lý khỏi database. Dữ liệu có thể khôi phục.
Software Requirements Specification	SRS	Tài liệu đặc tả yêu cầu phần mềm theo IEEE 830 / ISO/IEC/IEEE 29148.
TanStack Query	—	Thư viện React quản lý server state: caching, background refetch, optimistic updates.
tsvector / tsquery	—	Kiểu dữ liệu PostgreSQL cho full-text search. tsvector là chỉ mục đã xử lý, tsquery là biểu thức tìm kiếm.
Unit of Work	UoW	Pattern đảm bảo nhiều operations được thực hiện trong một transaction duy nhất.